**AI420 Agentic AI**
**HOS01 – Experiencing a Basic AI Agent**
07/07/2025 Developed by Sam Chung
07/18/2025 Reviewed by Clark Ngo
03/30/2025 Revised by Sam Chung

This HOS introduces the simplest form of ADK agents. You will learn how to create a basic agent that can respond to user queries.

**Before You Start**

- **Screenshots may be different from your environment.**
- The directory path shown in screenshots may be different from yours.
- The steps might have subtle discrepancies. Please use your best judgment while completing each step in this cookbook-style tutorial.
- Some steps may not be explained in detail. If you are not sure what to do:
  1. Consult the resources from the course.
  2. If you cannot solve the problem after a few tries (usually 15 -30 minutes), ask a TA for help.

**Readings and Examples:**

- Visit the Repository for Examples.
  - Select the related module.
  - Visit the README.md file.
  - Find examples for your practices.

**Learning Outcomes**

- Section 1: Accessing GitHub Codespaces
- Section 2: Installing Python Extension
- Section 3: Setting up a Virtual Environment
- Section 4: Creating an Agent Project
- Section 5: Setting up an External LLM Service
- Section 6: Running Your Agent using Text
- Section 7: Running Your Agent using Voice
- Section 8: Pushing Your Work to GitHub

**Upload the following to your GitHub Repository generated from GitHub Classroom.**

- Take a screenshot 'Section03_virtual_environment_firstname_lastname.png' by using your first and last name.
- Take a screenshot 'Section04_root_agent_firstname_lastname.png' by using your first and last name.
- Take a screenshot 'Section05_root_agent_firstname_lastname.png' by using your first and last name.

- Take a screenshot 'Section06_Trace_firstname_lastname.png' by using your first and last name.
- Take a screenshot 'Section06_STRIDE_firstname_lastname.png' by using your first and last name

**Resources**

- Google Gemini, GitHub Copilot, and Grammarly were used to develop this HOS.
- Google Agent Development Kit (ADK)
- google-adk·PyPI
- Gemini models | Gemini API | Google AI for Developers
- Brandon Hancock's ADK Crash Course
  - Module 1: Basic Agent
- Brandon Hancock's ADK Masterclass: Build AI Agents & Automate Workflows (Beginner to Pro) (3:12:47)
  - Module 1: Basic Agent (0:04:20 – 0:23:00)

**Key Concepts and Tools for Experiencing a Basic Agent**

- AI Agent or LLM Agent (LlmAgent)
- Google Agent Development Kit (ADK)
- Google LLMs
- Google Cloud Console
- Google API Key
- Google Ai Studio
- System Architecture
- Unified Modeling Language (UML) Deployment Diagram

An "**AI Agent or LLM Agent**" is a sophisticated AI system that uses a Large Language Model (LLM) as its core "brain" to achieve complex goals. It goes beyond a simple LLM that just responds to a single prompt.

Here's a breakdown of what makes an LLM Agent:

1. Large Language Model (LLM) as the Brain: At its heart, an LLM agent uses a powerful LLM (like Gemini, GPT-4, etc.) to understand human language, reason through problems, generate responses, and make decisions. The LLM processes the user's input and internal thoughts, acting as the central coordinator.
2. Reasoning and Planning: Unlike a basic LLM, an agent can often:-
   - Reason: Think through a problem, break it down into smaller steps, and draw logical conclusions. This often involves techniques like "chain-of-thought" prompting.
   - Plan: Formulate a sequence of actions needed to achieve a specific goal, even for multi-step tasks.
3. Memory: LLM agents incorporate memory to maintain context and learn over time:
   - Short-term memory: This allows the agent to remember the immediate conversation history and current context within a single interaction. It's like a notepad for the ongoing task.
   - Long-term memory: This enables the agent to recall information from past interactions, learn from previous experiences, and adapt its behavior to user preferences or common patterns. This often involves storing information in vector databases and retrieving relevant snippets.
4. Tools and Interaction with the Environment: A crucial aspect of LLM agents is their ability to use external "tools." These tools extend the agent's capabilities beyond its inherent language generation:

- Information Retrieval: Accessing databases, knowledge bases, or the internet (via search APIs) to gather specific, up-to-date information.
- Calculations/Logic: Using code interpreters, mathematical engines, or other APIs to perform computations or execute specific logic.
- External Systems: Interacting with real-world systems via APIs (e.g., scheduling appointments, sending emails, querying financial data).

5. Autonomous Operation (often): Many LLM agents are designed to operate with minimal human intervention. Given a high-level objective, they can plan, execute, observe results, and iterate until the goal is met. This "sense-think-act" loop allows them to be more proactive and capable than simple prompt-response systems.

How LLM Agents Work (Simplified Loop):

1. Perceive/Receive Input: The agent receives a user query or observes a change in its environment.
2. Think/Reason: The LLM (brain) analyzes the input, uses its reasoning capabilities to understand the goal, and formulates a plan. This might involve breaking down the task into sub-tasks.
3. Act/Execute: Based on its plan, the agent decides what action to take. This could be:
   - Generating a direct response.
   - Calling an external tool (e.g., performing a web search, running a Python script).
   - Delegating to another specialized agent in a multi-agent system.
4. Observe: The agent observes the outcome of its action (e.g., the result of a tool call, feedback from the environment).
5. Reflect/Iterate: The agent evaluates the observed outcome against its goal. If the goal isn't met, it reflects on what happened, adjusts its plan, and continues the loop until the task is completed or it determines it cannot proceed.

Key Benefits and Applications:
LLM agents are transforming various industries by enabling more intelligent, adaptable, and autonomous AI systems. They are used for:

- Customer Service: More sophisticated chatbots that can resolve complex issues, gather information, and even interact with internal systems.
- Content Generation: Generating detailed reports, marketing copy, code, or creative content that requires multi-step reasoning and external data.
- Workflow Automation: Automating multi-step business processes that previously required human intervention.
- Research and Analysis: Acting as research assistants, gathering and synthesizing information from various sources.
- Personalized Experiences: Providing highly personalized recommendations or assistance by remembering past interactions and preferences.

In essence, an LLM agent extends the raw power of a Large Language Model by giving it the ability to reason, remember, plan, and interact with the real world through tools, making it a much more capable and autonomous AI system.

The **Google Agent Development Kit (ADK)** is an open-source, code-first framework designed to simplify the process of building, evaluating, and deploying sophisticated AI agents, particularly those powered by large language models (LLMs). In essence, the Google ADK empowers developers to build sophisticated,

modular, and production-ready AI agents, from simple task executors to complex multi-agent systems, with a focus on flexibility, control, and seamless integration within the Google Cloud ecosystem.

1. Purpose and Core Philosophy:
   - Simplifying Agent Development: ADK aims to make building complex AI agents more akin to traditional software development, focusing on code-first definitions, modularity, and testability.
   - Controllable Agent Behavior: It provides developers with precise control over agent behavior and orchestration, which is crucial for building reliable and predictable AI applications.
   - Multi-Agent Systems: A core strength of ADK is its design for building and managing multi-agent systems, where specialized agents can collaborate, delegate tasks, and operate in a hierarchical structure to solve complex problems.
2. Key Features and Capabilities:
   - Code-First Development (Python & Java): Developers define agent logic, tools, and orchestration directly in Python (and recently Java), offering maximum flexibility and integration with existing software development practices.
   - Rich Tool Ecosystem:
     - Agents can be equipped with a diverse set of "tools" to interact with the outside world, such as pre-built Google tools (like Google Search, BigQuery), custom Python functions, OpenAPI specifications, and integration with third-party libraries (e.g., LangChain, CrewAI).
     - A significant part of this is the Model Context Protocol (MCP) Toolbox, which standardizes how agents connect to various data sources, including databases.
   - Flexible Agent Orchestration:
     - LLM Agents (LlmAgent): Utilize LLMs for natural language understanding, reasoning, planning, and dynamic decision-making.
     - Workflow Agents (SequentialAgent, ParallelAgent, LoopAgent): Define predictable execution flows for other agents (e.g., run agents in a sequence, in parallel, or in a loop) without relying on an LLM for flow control.
     - Custom Agents: Allow for highly tailored operational logic and integrations by extending the base agent class.
   - Integrated Developer Experience:
     - Local Testing & Debugging: Provides a powerful Command Line Interface (CLI) and a visual Web UI for developing, testing, and debugging agents locally. You can inspect events, state, and agent execution step-by-step.
     - Built-in Evaluation: Offers a framework to systematically assess agent performance against predefined test cases, evaluating both the final response quality and the execution trajectory.
   - Deployment Options:
     - Agents built with ADK can be easily containerized and deployed.
     - Optimized for seamless deployment on Google Cloud platforms like Cloud Run or scaled using Vertex AI Agent Engine, a fully managed runtime for production AI agents with built-in features for context management, infrastructure, scaling, security, and monitoring.
   - Bidirectional Streaming: Supports unique bidirectional audio and video streaming capabilities for human-like and interactive conversations, moving beyond just text-based interactions.17
   - Model Agnostic (with Google Optimization): While optimized for Gemini and the Google Cloud ecosystem (Vertex AI), ADK is designed to be model-agnostic, meaning it can work with various LLMs accessible via Model Garden or LiteLLM.
   - Agent-to-Agent (A2A) Protocol: Enhancements in communication protocols allow agents to collaborate effectively and securely, supporting stateless interactions and clear authentication

requirements.

3. Relationship with Other Google AI Offerings:

- Vertex AI Agent Engine: ADK provides the framework for building agents, and Agent Engine is the fully managed platform for deploying, managing, and scaling those agents in production.20 They work in tandem for a complete develop-to-deploy experience.
- Gemini Models: ADK is optimized for integration with Google's Gemini models, leveraging their advanced reasoning and tool-use capabilities.21
- Model Context Protocol (MCP): ADK supports MCP, which enables agents to connect to diverse data sources and capabilities by leveraging an ecosystem of MCP-compatible tools (including databases).

**<u>Google LLMs</u>**

The Gemini API offers different models that are optimized for specific use cases. Here's a brief overview of Gemini variants that are available:

| Model variant | Input(s) | Output | Optimized for |
|---|---|---|---|
| Gemini 2.5 Pro<br>`gemini-2.5-pro` | Audio, images, videos, text, and PDF | Text | Enhanced thinking and reasoning, multimodal understanding, advanced coding, and more |
| Gemini 2.5 Flash<br>`gemini-2.5-flash` | Audio, images, videos, and text | Text | Adaptive thinking, cost efficiency |
| Gemini 2.5 Flash-Lite Preview<br>`gemini-2.5-flash-lite-preview-06-17` | Text, image, video, audio | Text | Most cost-efficient model supporting high throughput |
| Gemini 2.5 Flash Native Audio<br>`gemini-2.5-flash-preview-native-audio-dialog` & `gemini-2.5-flash-exp-native-audio-thinking-dialog` | Audio, videos, and text | Text and audio, interleaved | High quality, natural conversational audio outputs, with or without thinking |
| Gemini 2.5 Flash Preview TTS<br>`gemini-2.5-flash-preview-tts` | Text | Audio | Low latency, controllable, single- and multi-speaker text-to-speech audio generation |
| Gemini 2.5 Pro Preview TTS<br>`gemini-2.5-pro-preview-tts` | Text | Audio | Low latency, controllable, single- and multi-speaker text-to-speech audio generation |
| Gemini 2.0 Flash<br>`gemini-2.0-flash` | Audio, images, videos, and text | Text | Next generation features, speed, and realtime streaming. |
| Gemini 2.0 Flash Preview Image Generation<br>`gemini-2.0-flash-preview-image-generation` | Audio, images, videos, and text | Text, images | Conversational image generation and editing |

The **Google Cloud Console** is the web-based graphical user interface (GUI) that serves as the central hub for managing all your resources and services within Google Cloud Platform (GCP).

The Google Cloud Console is a tool used for:

- Developers: To deploy applications, manage APIs, and integrate with various services.
- IT Administrators: To manage infrastructure, control access, monitor performance, and ensure security.
- Data Engineers and Scientists: To manage data pipelines, databases, and machine learning workflows.
- Business Users: To get an overview of their cloud spending and resource utilization.

A **Google API Key** is a unique alphanumeric string that serves as a credential to authenticate and authorize an application or project when it makes requests to Google APIs.

When your ADK agent needs to interact with Google's AI models, such as the Gemini API (especially when using Google AI Studio), an API key is typically required to authenticate your agent's requests. This allows your agent to send prompts, receive responses, and utilize the capabilities of these models.
ADK agents can leverage a wide range of Google services (e.g., Google Maps, Google Sheets, Gmail, etc.) to perform tasks. To access these services programmatically, your agent needs an API key with the appropriate permissions enabled.

A **Google AI Studio** is an interactive platform where you can quickly test out different prompts, see how the AI models respond, and iterate on your ideas without needing to write extensive code initially. Google Cloud Platform underpins it. Projects you create and API keys you generate are linked to your Google Cloud account, allowing for seamless transition to more advanced development workflows if needed.

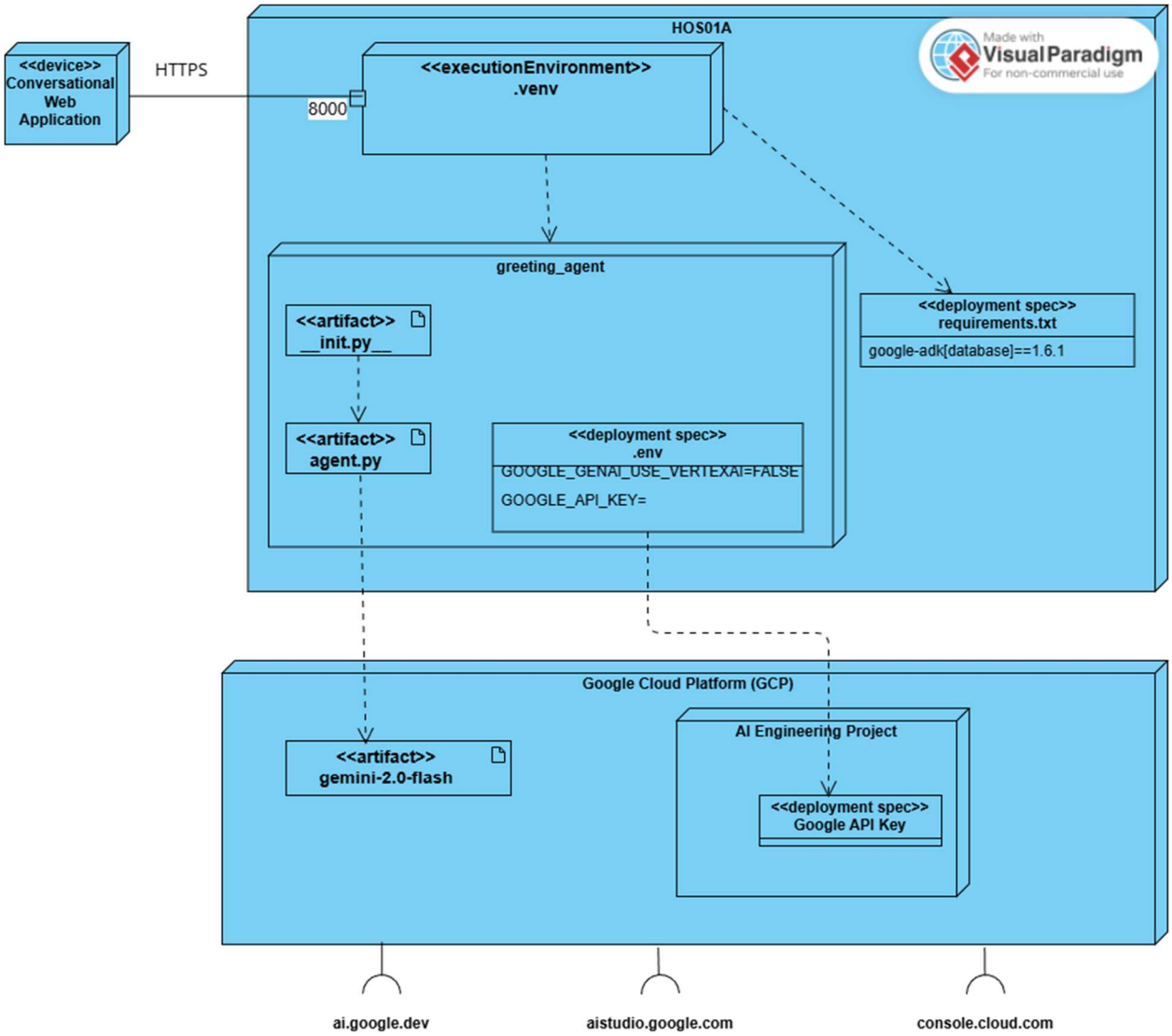
**System Architecture** is the fundamental design of a system, defining its structure, behavior, and various views. It's the high-level blueprint that guides the development process, ensuring all parts work together coherently to meet specific goals and non-functional requirements like scalability, security, and performance.

System architecture is often described through multiple "views" (or "architectural perspectives") to cater to different stakeholders and concerns. Common views include:

- Logical View (Conceptual/Design) - What it does.
- Process View (Concurrency/Distribution) - How it runs concurrently.
- Development View (Implementation/Module) - How it's built.
- Physical/Deployment View (Runtime/Hardware) - Where it runs physically)
- Scenario/Use Case View - How it's used.

A **UML Deployment Diagram** is a specific type of Unified Modeling Language (UML) diagram that visually represents the physical deployment of software artifacts on hardware nodes within a system. It provides a static, runtime view of the system's topology. It answers the question: "Where do the pieces of my software actually run in the physical world, and how are those physical machines connected?"

1. Nodes (The "Where")
2. Artifacts (The "What Runs"):
3. Deployment Specifications (The "How It Runs")
4. Communication Paths (The "How Connected")




**Google Gemini explained the given UML deployment diagram below. (The author modified some explanations for clarity.)**

This diagram illustrates how a "greeting_agent" application, an AI-powered agent, is deployed across different execution environments, specifically showing its interaction with Google Cloud Platform (GCP) services, including a Gemini LLM. It also highlights various configuration details and dependencies.

**Main Components and Relationships:**

1. Conversational Web Application (Node for client)
   - Description: Represents an external client-side web application (e.g., a browser-based frontend, a mobile app, or another service) that initiates interactions.
   - Connection: It connects to HOS01A using the HTTPS protocol, specifically targeting port 8000. This suggests HOS01A is an API endpoint or web server.
2. HOS01A (Top-Level Node):
   - Description: This is the primary logical grouping for your application's deployment. It represents the main server or environment where your greeting_agent resides.
   - Internal Components:

- .venv (Executable Node): This nested node represents a Python virtual environment. This indicates that greeting_agent runs within an isolated Python environment, which is a common best practice for dependency management.
  - ✓ Connection: A dashed line from .venv to greeting_agent's components (specifically agent.py and init.py) with a dashed arrow indicates that these artifacts are deployed *within* or *run on* this virtual environment.
- greeting_agent (Node/Component): This is the core application logic. It's depicted as a node containing artifacts.
  - ✓ __init.py__: An artifact representing an initialization script or module. The underscore might indicate it's a "dunder" (double underscore) file, often used for package initialization in Python.
  - ✓ agent.py: The main Python script artifact that contains the core logic of your "greeting agent."
  - ✓ .env: This is a deployment specification for environment variables.
    - ➢ GOOGLE_GENAI_USE_VERTEXAI=FALSE: This is crucial. It tells the Google Generative AI client library (likely used by your agent) *not* to use Vertex AI for LLM access. This implies it's either using the standard Gemini API directly or a local setup for development/testing.
    - ➢ GOOGLE_API_KEY=: This indicates that an API key is required, but its value is expected to be provided via this environment variable. The blank value signifies it's a placeholder.
  - ✓ requirements.txt: This artifact specifies the Python package dependencies (e.g., google-adk[database]==1.6.1). This confirms the use of Google's Agent Development Kit (ADK) and specifies a particular version. This deployment specification is associated with the .venv execution environment, meaning these requirements are installed within it.

3. Google Cloud Platform (GCP) (Bottom-Level Node):
   - Description: This represents the entire Google Cloud Platform environment, which is where external Google services are hosted and accessed.
   - Internal Components:
     - AI Engineering Project (Node/Component): This is a nested node representing a specific Google Cloud Project, which serves as a logical container for all resources, billing, and permissions within GCP.
       - ✓ Google API Key: This deployment specification is located within the "AI Engineering Project." This signifies that the Google API Key itself is managed and resides within this specific GCP project. This is where the actual API key string (the credential) would be configured for your project.
     - gemini-2.0-flash: This is an artifact representing the specific Gemini Large Language Model (LLM) version being used.
       - ✓ Deployment: It's shown directly under the "Google Cloud Platform (GCP)" node, implying it's a managed service/artifact provided by GCP.
       - ✓ Connection: A dashed line from agent.py to gemini-2.0-flash indicates that the agent.py script interacts with or consumes the gemini-2.0-flash model. This interaction is typically via an API call (e.g., the Gemini API).
4. External Google URLs (Bottom Labels):
   - ai.google.dev: This is the general developer portal for Google AI. No direct connection is shown, but it serves as the source for documentation and resources related to the LLM and ADK.

- aistudio.google.com: This is Google AI Studio, a web-based environment for prototyping and interacting with generative AI models. Again, no direct connection, but conceptually related to where one might develop or test agents.
- console.cloud.com: This is the Google Cloud Console, where the "AI Engineering Project" and its associated Google API Key are managed. This aligns with the AI Engineering Project and Google API Key elements in the diagram.

**Flow of Interaction and Execution:**

1. A Web Application sends an HTTPS request to port 8000 on the HOS01A server.
2. The HOS01A server's environment (specifically the .venv) receives this request.
3. The greeting_agent's init.py and agent.py scripts are executed within the .venv.
4. The agent.py script, leveraging the google-adk library and using the GOOGLE_API_KEY environment variable for authentication, makes API calls to interact with the gemini-2.0-flash LLM.
5. The gemini-2.0-flash LLM processes the prompts from the agent (this processing happens within Google's cloud infrastructure, managed by services like Vertex AI, even though GOOGLE_GENAI_USE_VERTEXAI=FALSE implies direct Gemini API access rather than Vertex AI's specific managed endpoints).
6. The LLM returns responses to agent.py.
7. The agent.py processes the LLM's response and likely sends a greeting or other appropriate output back to the Web Application.

**Key Takeaways:**

- Hybrid Deployment: The diagram shows a hybrid deployment where the application logic (greeting_agent) is hosted in a custom environment (HOS01A) but relies on external managed Google Cloud services (specifically the Gemini LLM).
- Environment Variables for Configuration: It correctly highlights the use of environment variables to configure the agent's behavior and provide credentials.
- API Key Management: It visually represents that the Google API Key is a resource managed within a specific Google Cloud Project.
- Framework: The requirements.txt indicates the specific google-adk library version.
- LLM Access Method: The GOOGLE_GENAI_USE_VERTEXAI=FALSE is an interesting detail, suggesting a choice in how the Gemini API is consumed (direct access vs. via Vertex AI's higher-level features).

**Threat Modeling**

In traditional software, security is about keeping unauthorized users out. In Agentic AI, security focuses on preventing the agent from being manipulated into unsafe behavior.

- **What are we building?** (Scope: Creating a Data Flow Diagram or architectural map).
- **What can go wrong?** (Identification: Using frameworks like **STRIDE**).
- **What are we going to do about it?** (Mitigation: Implementing security controls like encryption or validation).
- **Did we do a good job?** (Validation: Testing if the fixes actually work).

**STRIDE**

| Threat | Security Goal | Agentic AI Specific Example |
|---|---|---|
| Spoofing | Authenticity | A malicious agent masquerading as a "Boss Agent" to give unauthorized commands to worker agents. |
| Tampering | Integrity | Indirect Prompt Injection: An attacker hides instructions in a document that the agent reads, tricking the agent into changing its plan or "Goal Hijacking." |
| Repudiation | Non-repudiability | An agent performs a sensitive action (like deleting a database record) but the system fails to log which agent did it or why. |
| Information disclosure | Confidentiality | An agent accidentally "leaks" its internal system prompt, API keys, or private user data during a conversation with an untrusted party. |
| Denial of service | Availability | Resource Exhaustion: An attacker triggers a recursive loop (e.g., Agent A calls Agent B, which calls Agent A) until the system runs out of memory or API credits. |
| Elevation of rivilege | Authorization | An agent with "Read-Only" access finds a way to use a "Write" tool, or a user tricks a high-privilege agent into acting on their behalf without verification. |

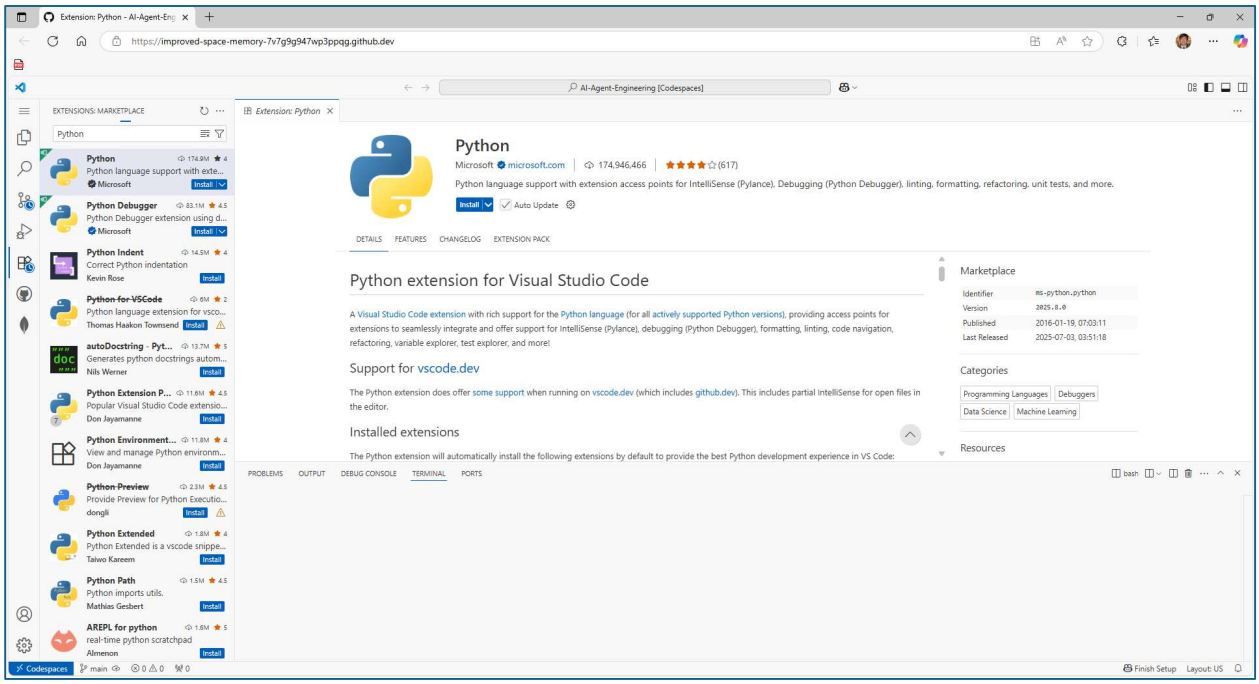
## Section 1: Accessing GitHub Codespaces

GitHub Codespaces are online cloud-based development environments that allow you to easily write, run, and debug your code. It is fully integrated with your GitHub repository and provides a seamless experience for developers.

1) Create your GitHub account if you do not have it.
2) Follow the instructions here.

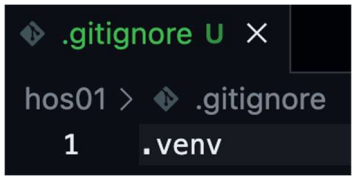
## Section 2: Installing Python Extension

1) Click Extensions in the left panel.
2) Search Python in the marketplace.
3) Click Install.



**Section 3: Setting up a Virtual Environment**

1) Before creating our venv (Python virtual environment), we need to create **.gitignore** to avoid accidentally committing unnecessary or bulky files.
   **>>> echo .venv >> .gitignore**




   means: append .venv to the .gitignore file
   Breakdown:
   - **echo '.venv'**

   Outputs the text .venv
   - **>> .gitignore**

   Appends that text to the end of the .gitignore file (creates it if it doesn't exist)

   **This tells Git to ignore the .venv/ directory, which is often used to store a Python virtual environment.**

   The **venv** directory can be **hundreds of megabytes**, and it's **not meant to be committed to Git**. If you don't ignore it, it may get staged and pushed to GitHub or your repo, bloating your repo size and causing unnecessary conflicts across environments.

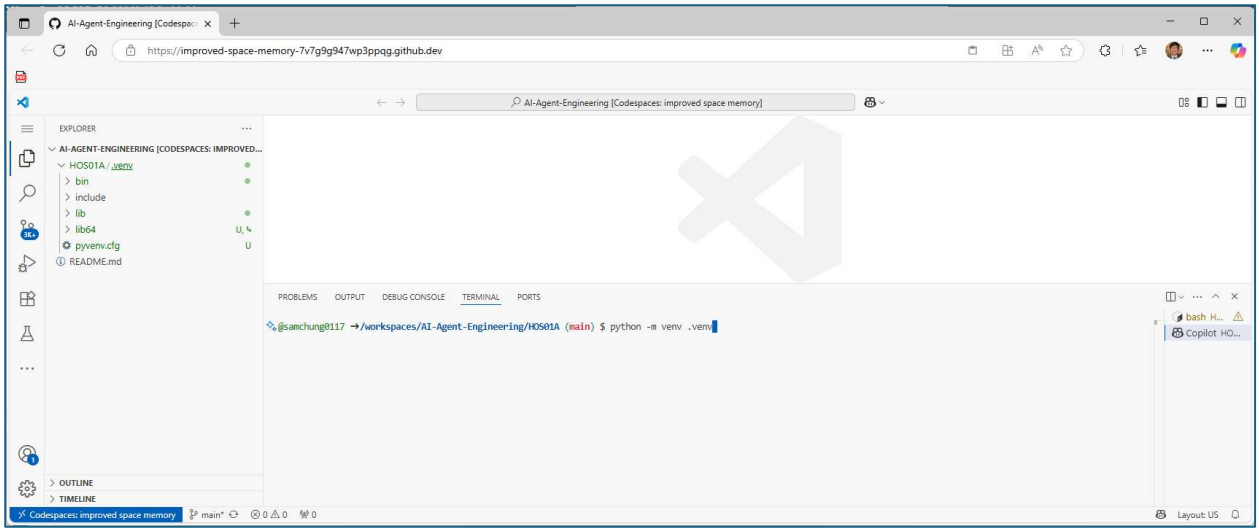
2) Create a virtual environment by using the Bash command below.
   **>>> python -m venv .venv**

   It creates a venv/ folder containing:
   - Python interpreter binaries
   - Standard library copies
   - Installed packages

   OS-specific folders (e.g., Scripts/, bin/, lib/)

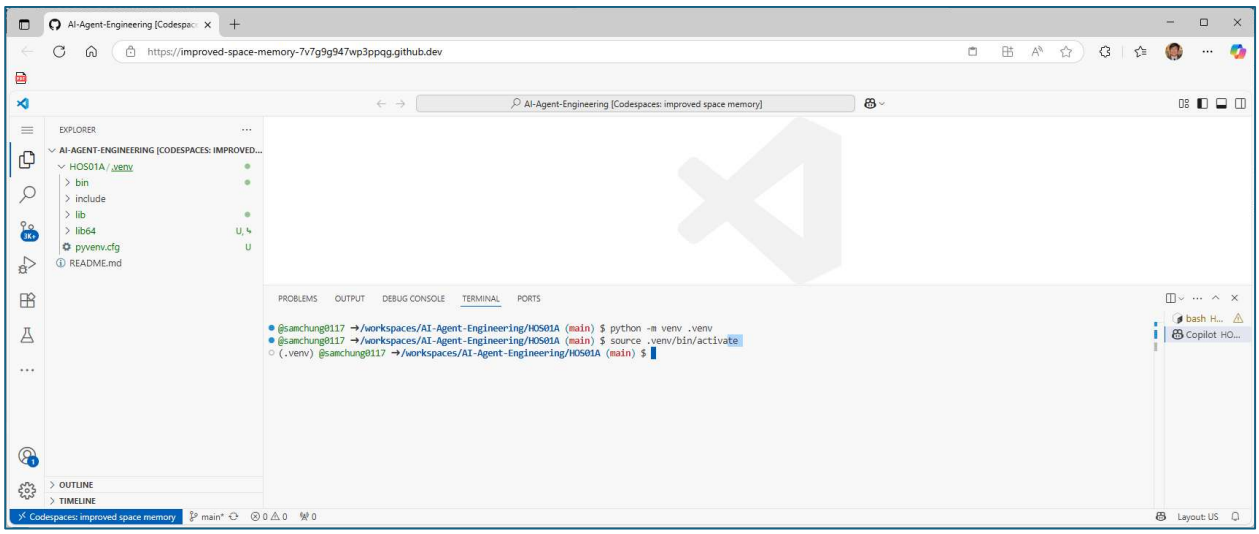
   - python: Invokes the Python interpreter.
   - -m venv: Tells Python to run the venv module, which is used for creating virtual environments.
   - .venv: Specifies the name of the directory where the virtual environment will be created. The leading dot (.) often indicates that it's a hidden directory, but its visibility depends on your operating system's settings.

   After running this command, a new directory named ".venv" will be created inside your project folder. This directory will contain an isolated Python installation and its own set of packages, keeping them separate from your system's global Python installation.


AI-Agent-Engineering [Codespac
https://improved-space-memory-7v7g9g947wp3ppqg.github.dev
AI-Agent-Engineering [Codespaces: improved space memory]
EXPLORER
AI-AGENT-ENGINEERING [CODESPACES: IMPROVED...
HOS01A / .venv
bin
include
lib
lib64
pyvenv.cfg
README.md
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
@samchung0117 ➜ /workspaces/AI-Agent-Engineering/HOS01A (main) $ python -m venv .venv
bash H...
Copilot HO...
Codespaces: improved space memory
main*
Layout: US

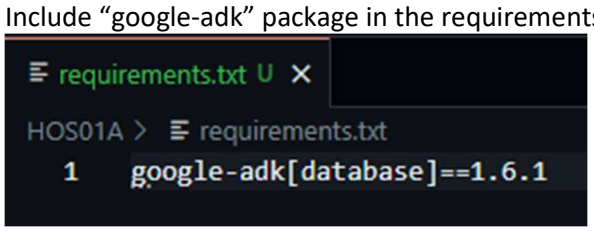
3) Activate the virtual environment. If you use GitHub Codespaces for cloud-hosted development environments, the default environment is created from an Ubuntu Linux image.
# macOS/Linux:
**source .venv/bin/activate**
This command instructs the current terminal session to use the specific Python interpreter and packages associated with the virtual environment located at .venv.

# Windows Command Prompt (CMD):
.venv\Scripts\activate.bat
# Windows PowerShell:
.venv\Scripts\Activate.ps1




4) Install Google Agent Development Kit (ADK).
   a. Create a “requirements.txt” file.
   **>>> echo google-adk[database]==1.6.1 >> requirements.txt**
   Create an empty file named requirements.txt and add the text: **google-adk[database]==1.6.1.** If the file requirements.txt already exists, it will add the same text (delete the duplicate text if necessary).

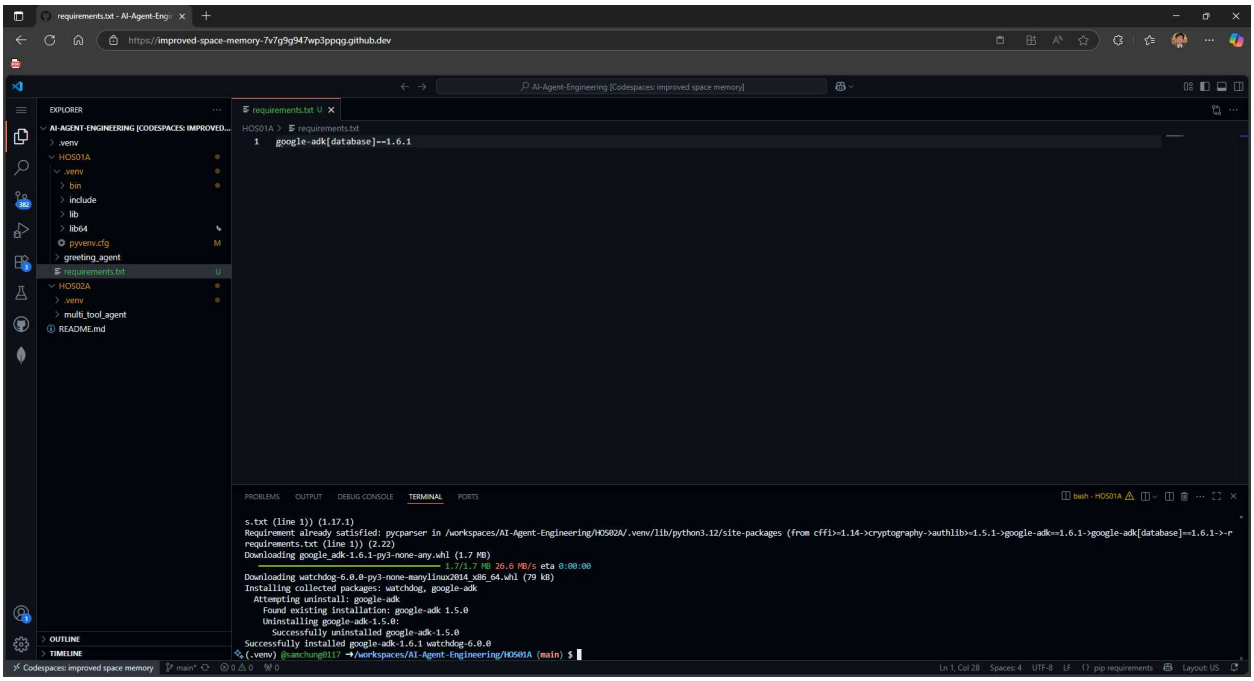
   b. Include “google-adk” package in the requirements.txt




   - google-adk: The main package for the Google Agent Development Kit.
   - [database]: This is an "extra" dependency. It tells pip to install not only the core google-adk package but also any additional packages required for its database integration features. These usually include database drivers or client libraries that google-adk can leverage to connect to various database systems (e.g., PostgreSQL, MySQL, BigQuery, etc., often via the Model Context Protocol (MCP) Toolbox).

- ==1.6.1: This specifies the exact version of google-adk that you want to install. This is crucial for ensuring reproducible environments.

c. pip reads the “requirements.txt” file and installs all the listed packages with the specified versions. “-r” is short for “—requirement”
**>>> pip install -r requirements.txt**



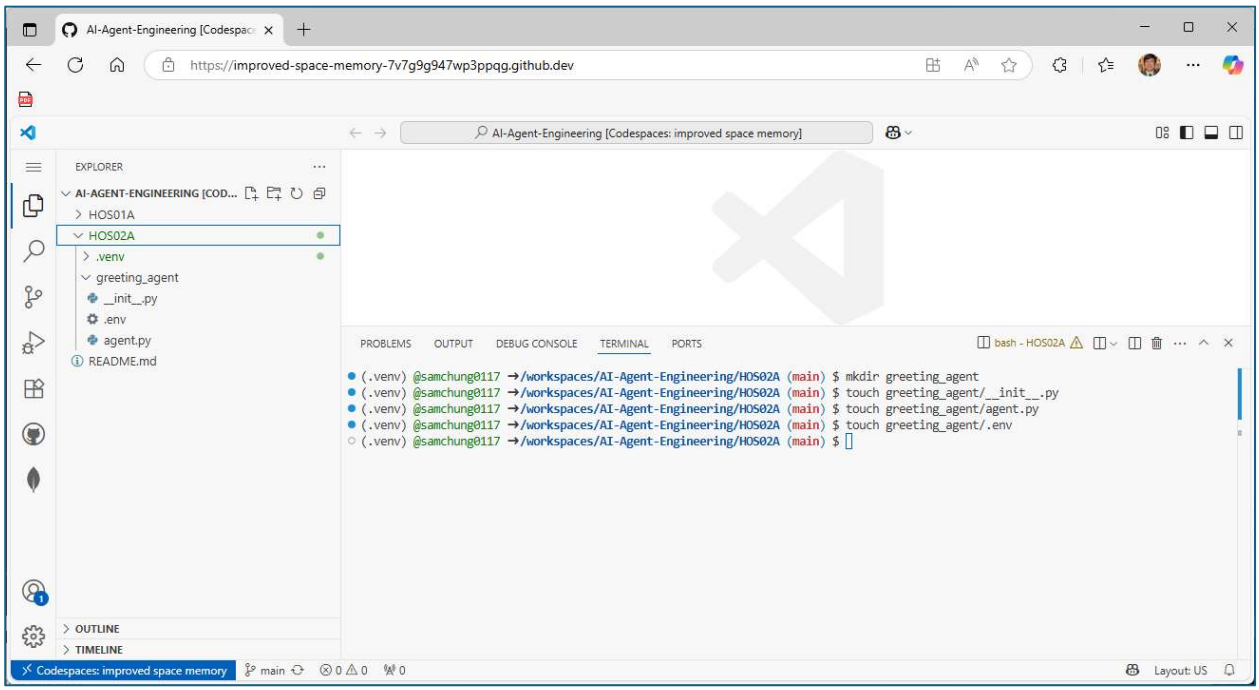

5) Take a screenshot ‘Section03_virtual_environment_firstname_lastname.png’ by using your first and last name.

**Section 4: Creating an Agent Project**

1) The following project structure will be used for an agent.

```
parent_folder/
    agent_folder/          # This is your agent's package directory
        __init__.py        # Must import agent.py
        agent.py           # Must define root_agent
        .env               # Environment variables
```

a) >>> mkdir greeting_agent
b) >>> touch greeting_agent/__init__.py
c) >>> touch greeting_agent/agent.py
d) >>> touch greeting_agent/.env




2) Copy and paste the following code into "__init__.py".
The Python statement "from . import agent" is a relative import.
- from .: Look within the current package (the package that the current Python file belongs to).
- import agent: Import a module or a name called agent from within that current package.

```
1   from . import agent
```

3) Copy and paste the following code into “agent.py”.

```
from google.adk.agents import Agent

root_agent = Agent(
    name="greeting_agent",
    # https://ai.google.dev/gemini-api/docs/models
    model="gemini-2.0-flash",
    description="Greeting agent",
    instruction="""
    You are a helpful assistant that greets the user.
    Ask for the user's name and greet them by name.
    """,
)
```

In essence, an LlmAgent is an intelligent orchestrator. You provide it with a brain (the model), a personality and mission (the instruction), and a set of hands (the tools). The LlmAgent then uses its LLM to reason about user input, decide which tools to use (if any), perform actions, and generate coherent responses.

1. **from google.adk.agents import Agent**
   - This line imports the necessary Agent class from the google.adk.agents library. The **ADK (Agent Development Kit)** is a framework provided by Google that helps developers build and manage AI agents, often powered by their Gemini models.
2. **root_agent = Agent(...)**
   - This line creates an instance of an Agent and assigns it to the variable root_agent. This root_agent is essentially your AI assistant.
3. **name="greeting_agent"**
   - Required
   - A unique string identifier for the agent.
   - Crucial for internal operations, especially in multi-agent systems where agents might refer to or delegate tasks to each other.
   - Choose a descriptive name (e.g., customer_support_router, billing_inquiry_agent).
4. **model="gemini-2.0-flash"**
   - Required.
   - Specifies the underlying LLM that will power the agent's reasoning.
   - gemini-2.0-flash refers to a specific, highly efficient, and fast version of Google's Gemini family of models. It's optimized for tasks requiring quick responses. The commented URL https://ai.google.dev/gemini-api/docs/models points to where you can find documentation about various available models.
   - The choice of model impacts the agent's capabilities, cost, and performance.
5. **description="Greeting agent"**
   - Optional
   - A concise summary of the agent's capabilities.
   - This provides a **brief, human-friendly description** of what the agent does. This is useful for documentation and for other developers (or even other AI agents) to understand its purpose at a glance.

- **Highly recommended for multi-agent systems**: Other LlmAgents use this description to determine if they should route a task to this agent. Make it specific to differentiate it (e.g., "Handles inquiries about current billing statements" rather than just "Billing agent").

6. **instruction=""" ... """"**
   - Required
   - This is the **core instruction (or system prompt)** that defines the agent's behavior and personality. It tells the LLM what its role is and how it should interact.
   - **The most critical parameter** for shaping an LlmAgent's behavior.
   - This is the "system prompt" or "persona" given to the LLM. It defines:
     - Its core task or goal (e.g., "You are a customer support agent.").
     - Its personality or persona (e.g., "Be polite and empathetic.").
     - Guidelines for when and how to use available tools.
     - Constraints or boundaries (e.g., "Do not share personal identifiable information.").
     - What to do if it can't handle a request (e.g., "Escalate to a human if unsure.").
   - In this case, the instruction is:
     - "You are a helpful assistant that greets the user." (Defines its role and helpful persona.)
     - "Ask for the user's name and greet them by name." (Specifies the exact task: prompt for the name, then use it in the greeting.)

**What This Agent Will Do**

When this root_agent is invoked, it will leverage the gemini-2.0-flash model to follow the given instructions. It will likely:

1. Initiate a conversation by asking the user for their name.
2. Wait for the user's response.
3. Once the user provides their name, it will use that name to deliver a personalized greeting.

6) Take a screenshot 'Section04_root_agent_firstname_lastname.png' by using your first and last name.

**Section 5: Setting up an External LLM Service**

The Google API Key is the fundamental mechanism that allows your AI agent to **authenticate** with Google's backend services, gain **authorized access** to the powerful LLMs, enable **billing and usage tracking**, and ensure adherence to **rate limits and quotas**. It's a non-negotiable component for developing and deploying AI agents that leverage Google's AI capabilities.

1) When using Python, open the **.env** file located inside (multi_tool_agent/) and copy-paste the following code from the Repository for Examples.

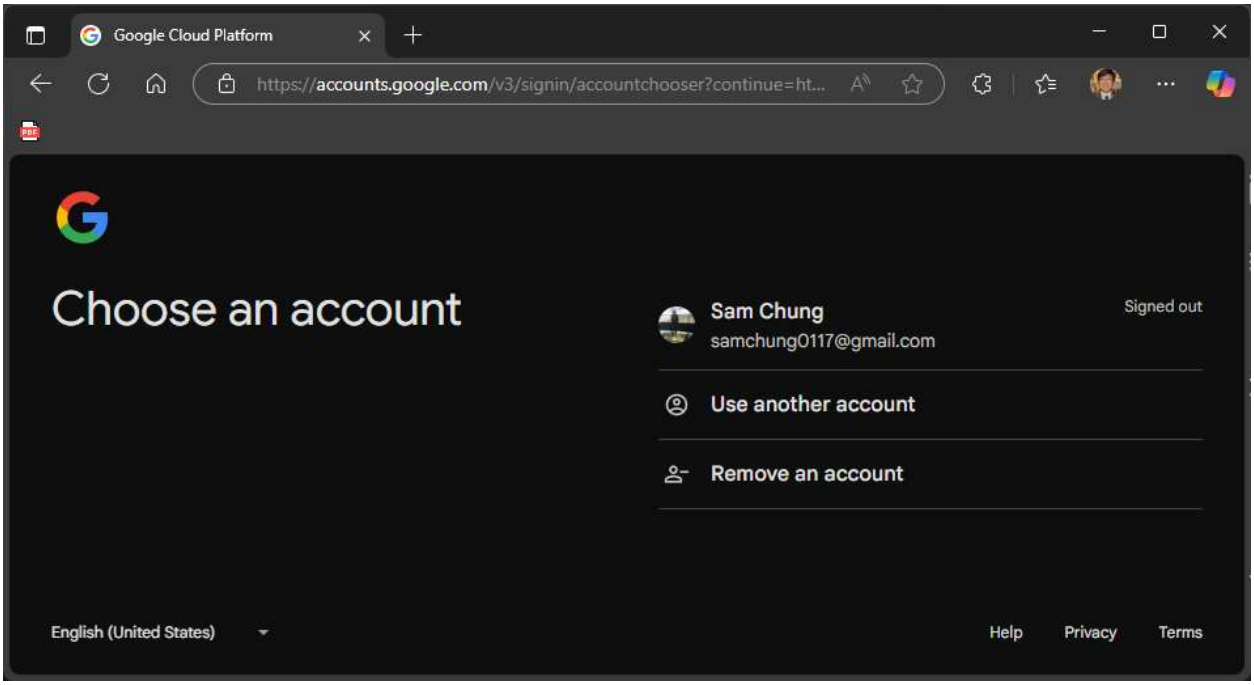
```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=PASTE_YOUR_ACTUAL_API_KEY_HERE
```

These lines are crucial setup steps to configure your Python application to correctly communicate with Google's AI models, ensuring it knows which endpoint to use and how to authenticate.

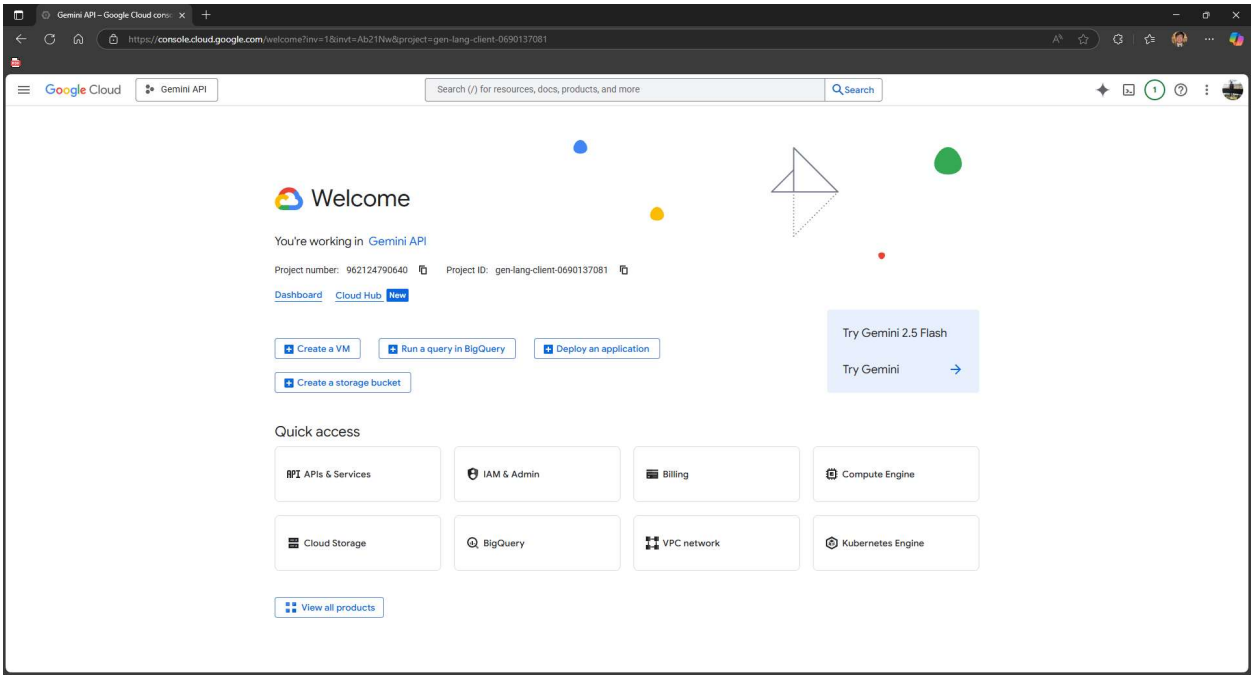
1. **GOOGLE_GENAI_USE_VERTEXAI=FALSE**
   - **Purpose:** This environment variable controls whether the Google Generative AI client library (which is what google.adk likely uses under the hood) should connect to the **Vertex AI** endpoint or the **Google AI Studio/Generative AI API** public endpoint.
   - **FALSE:** Setting it to FALSE (or 0, no, etc.) means you want to use the public Google AI Studio/Generative AI API (often for free tier usage or simpler deployments). If it were set to TRUE, it would try to connect to Vertex AI, which is Google Cloud's platform for building and deploying AI models, and usually requires a Google Cloud project and authentication.
2. **GOOGLE_API_KEY=your_API_key**
   - **Purpose:** This environment variable holds your **API key** for authenticating with Google services, specifically the Google Generative AI API.
   - **your_API_key:** This is a **placeholder**! You *must* replace your_API_key with an actual API key that you generate from the Google AI Studio or the Google Cloud Console.
   - **Security Note:** Never commit your actual API key directly into your code or public repositories. Using environment variables is a much safer practice for handling sensitive credentials.

**Creating a New Project**

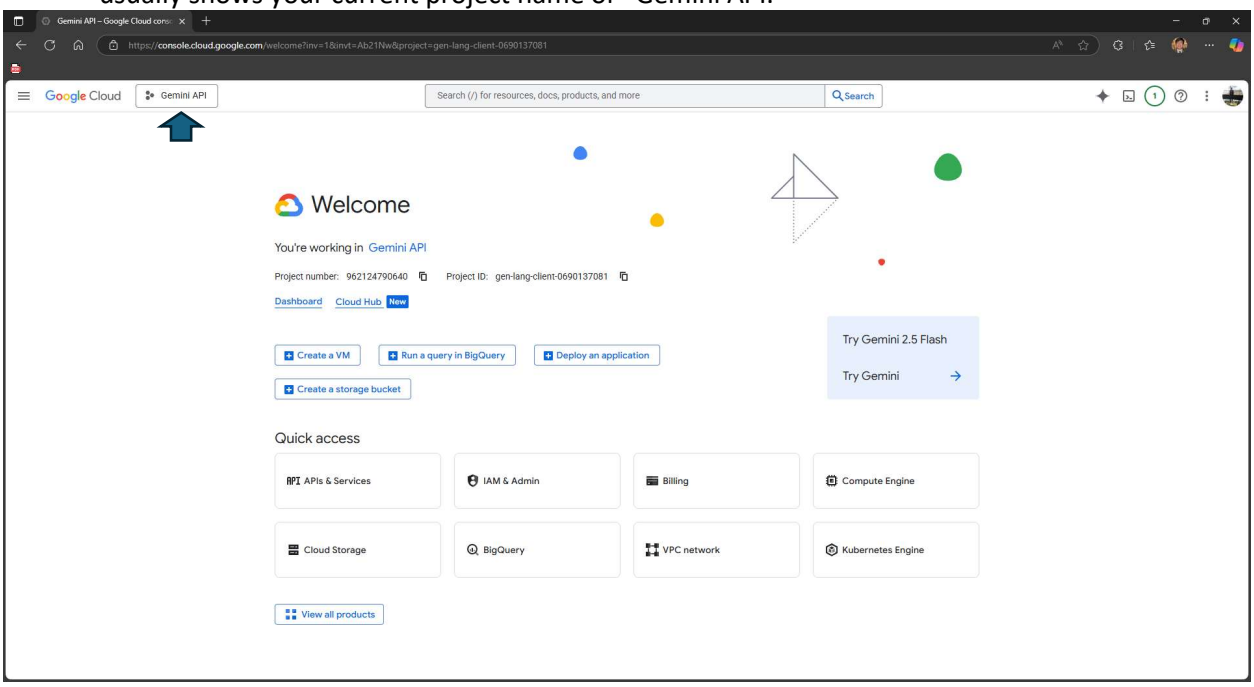
2) Go to the Google Cloud Console: Open your web browser and navigate to: https://console.cloud.google.com/



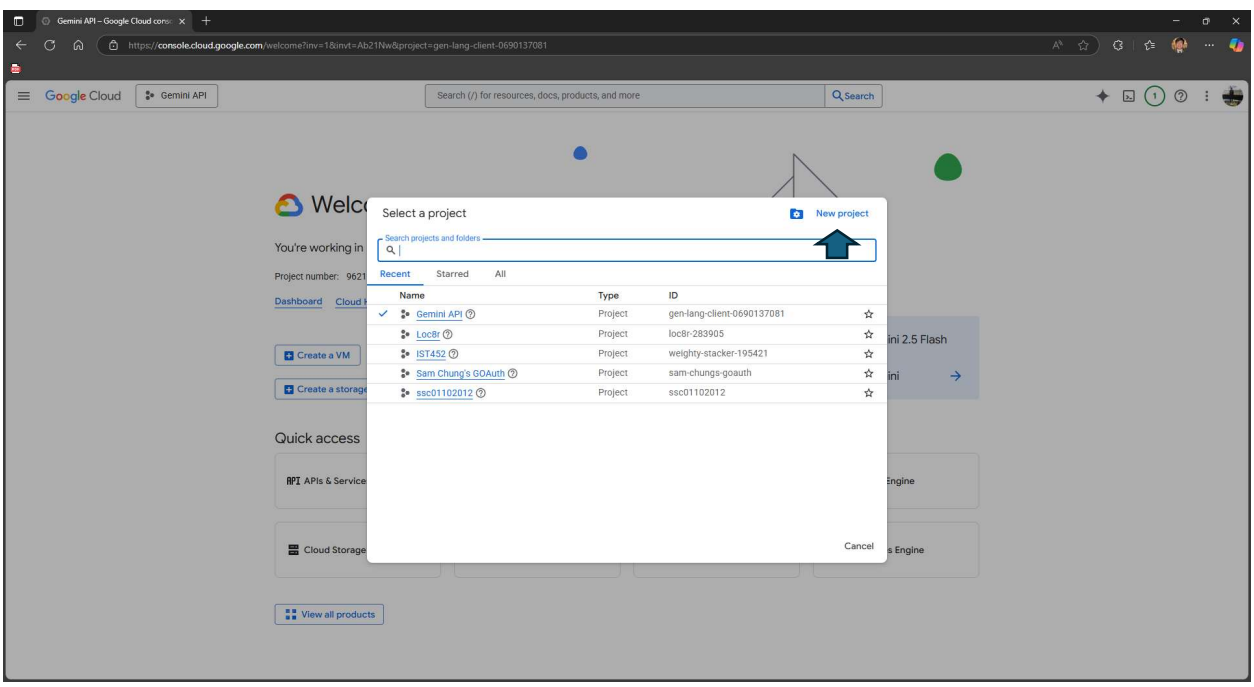

3) Sign In: If you're not already signed in, you'll be prompted to sign in with your Google account.



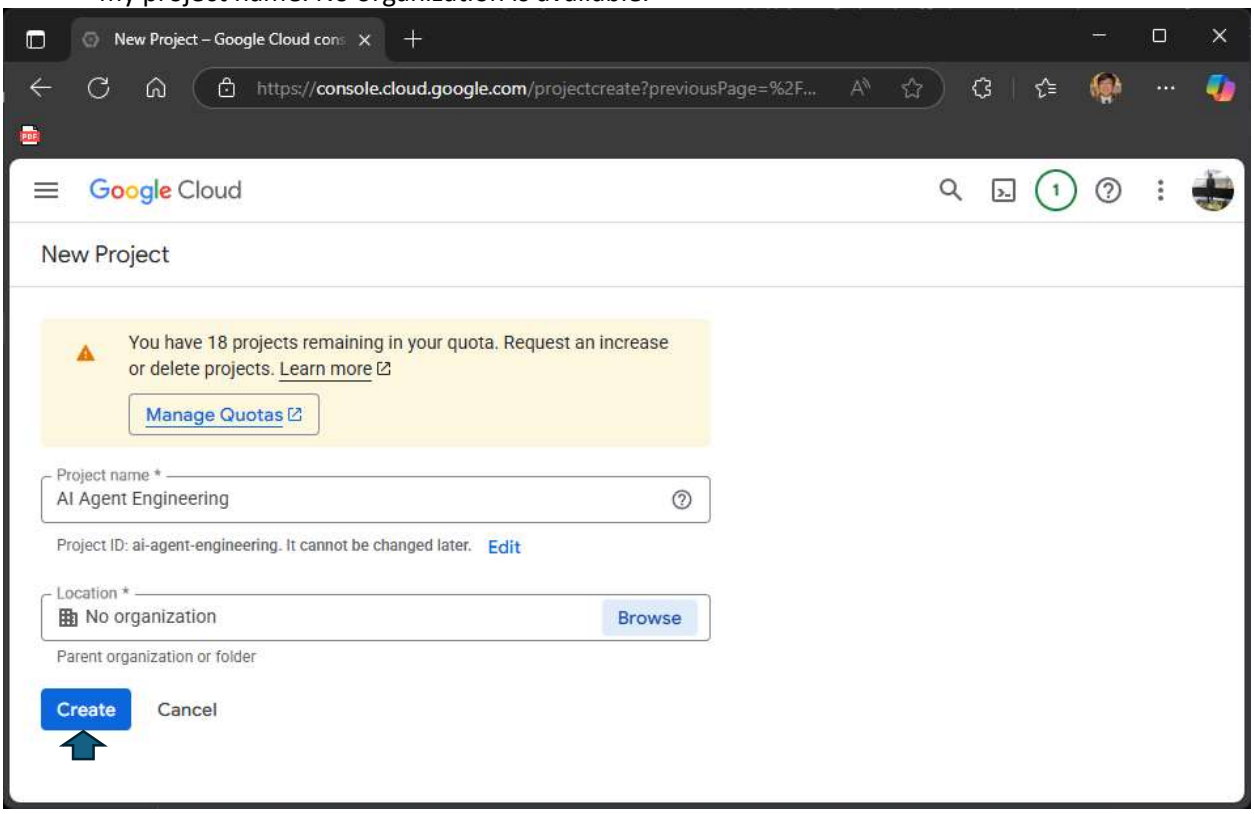
4) Create a New Project:
   a) Once in the console, look for the **project selector** dropdown at the very top of the page. It usually shows your current project name or "Gemini API."



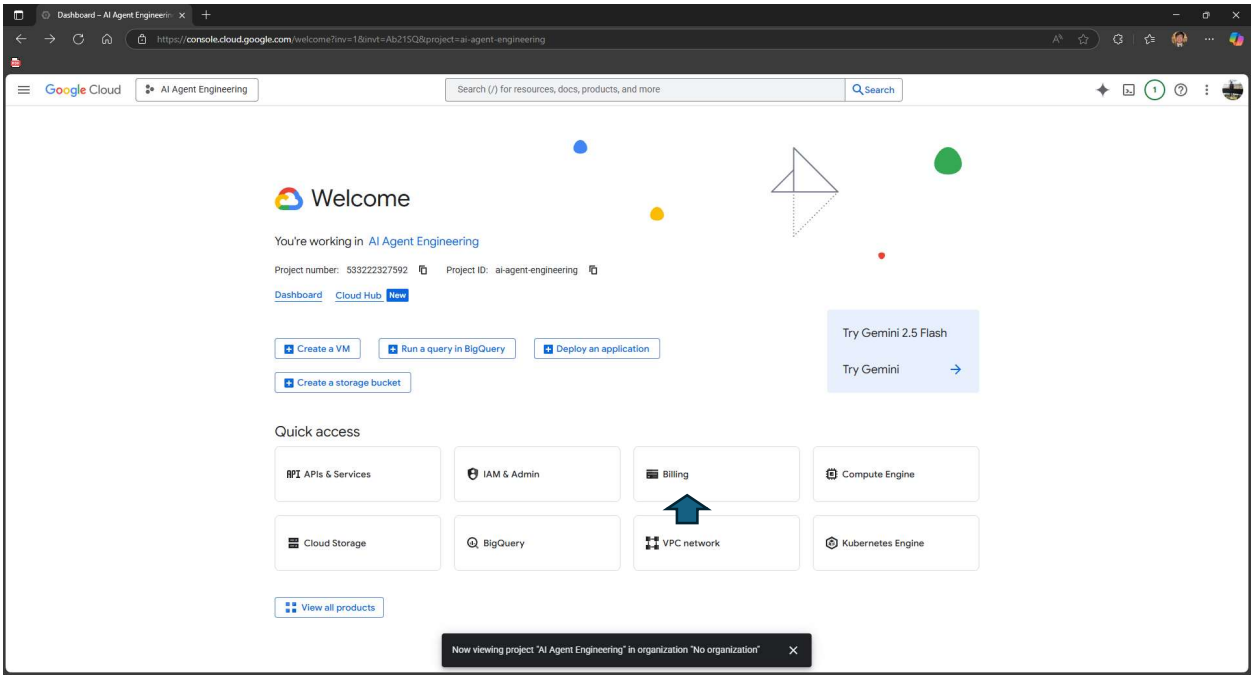

   b) In the "Select a project" dialog that appears, click the **"New Project"** button (it usually has a "+" icon).



c) Enter the project name and select the location. For example, I used “AI Agent Engineering” for my project name. No organization is available.

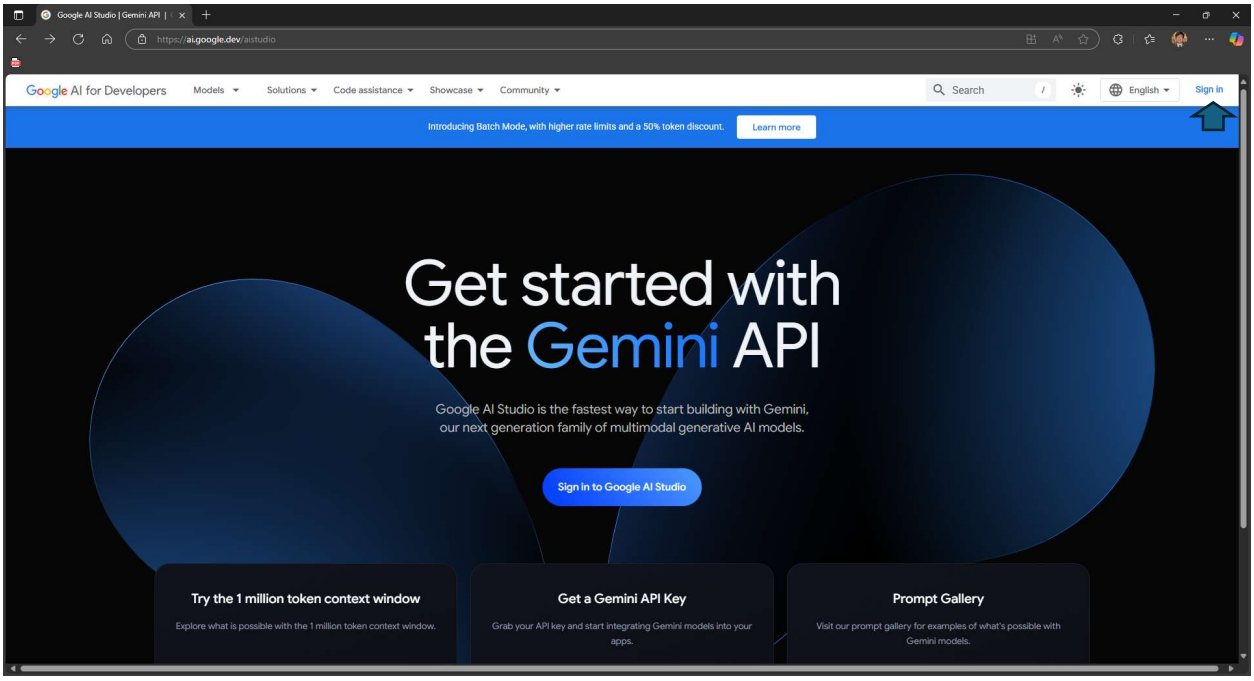



d) Create your billing account. **Please watch your Google Cloud Console for your pay since the use of Gemini API from a billing-enabled project is subject to <u>pay-as-you-go pricing</u>.**

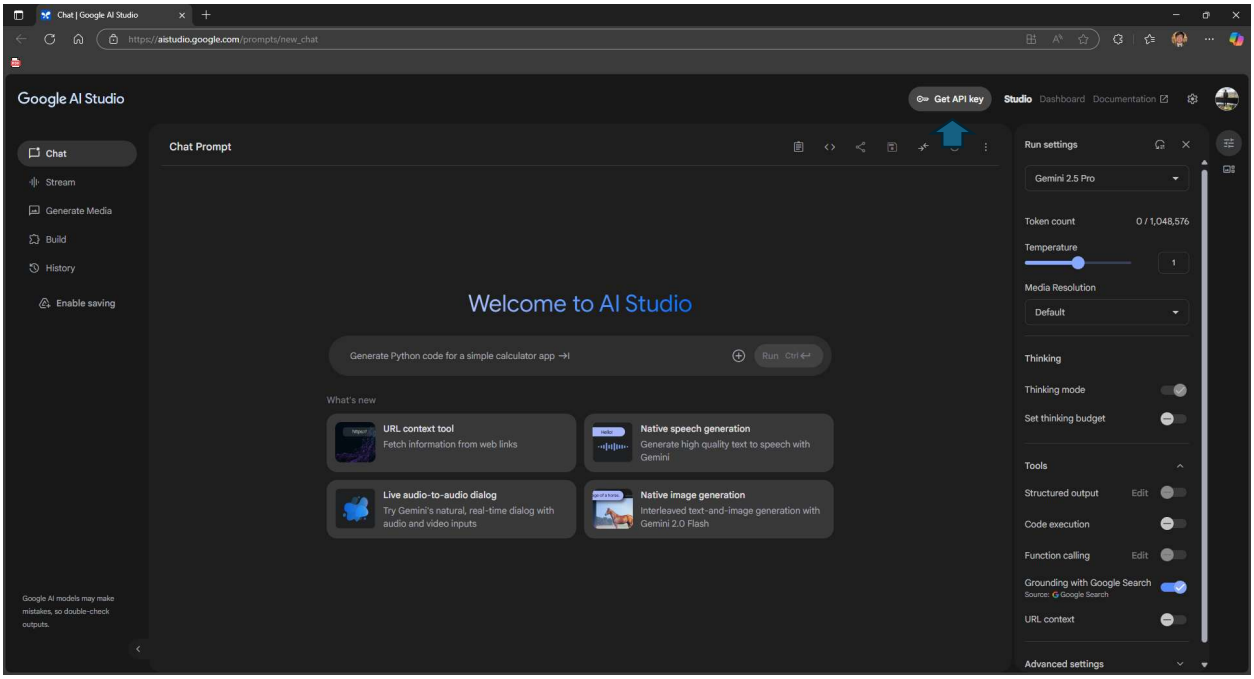


**Creating an API Key for the New Project**

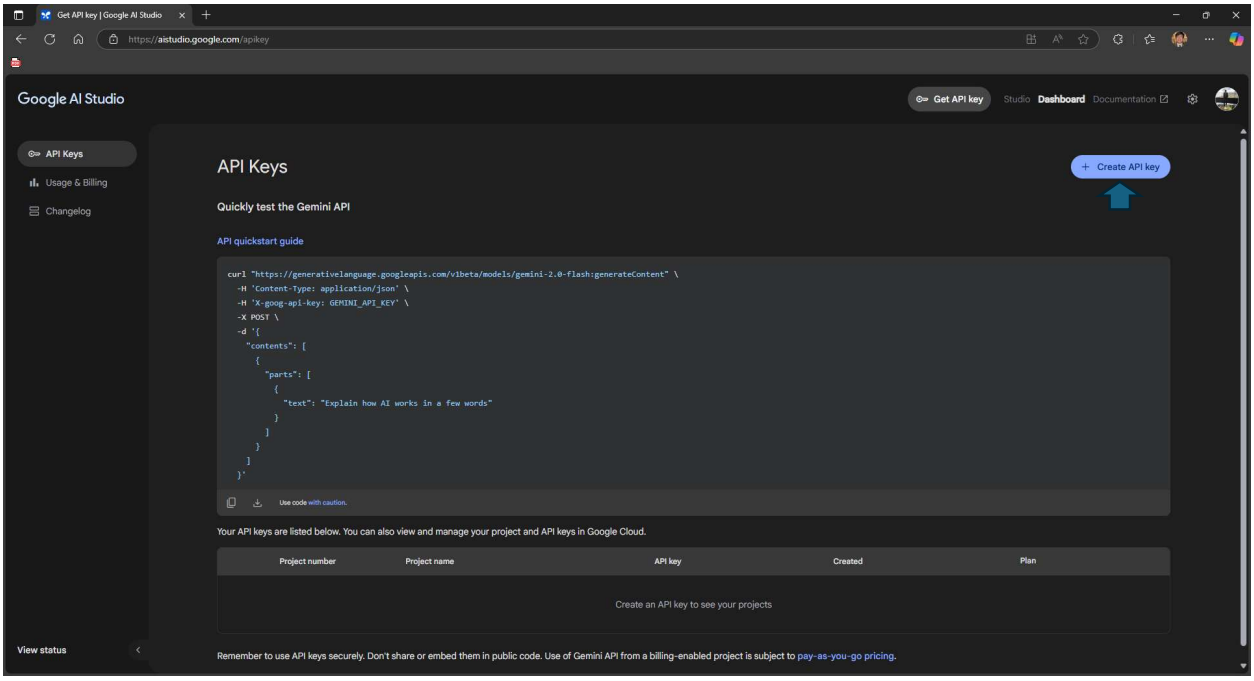
5) Visit Google AI Studio and log in by using the same account that you used for Cloud Cloude Console.



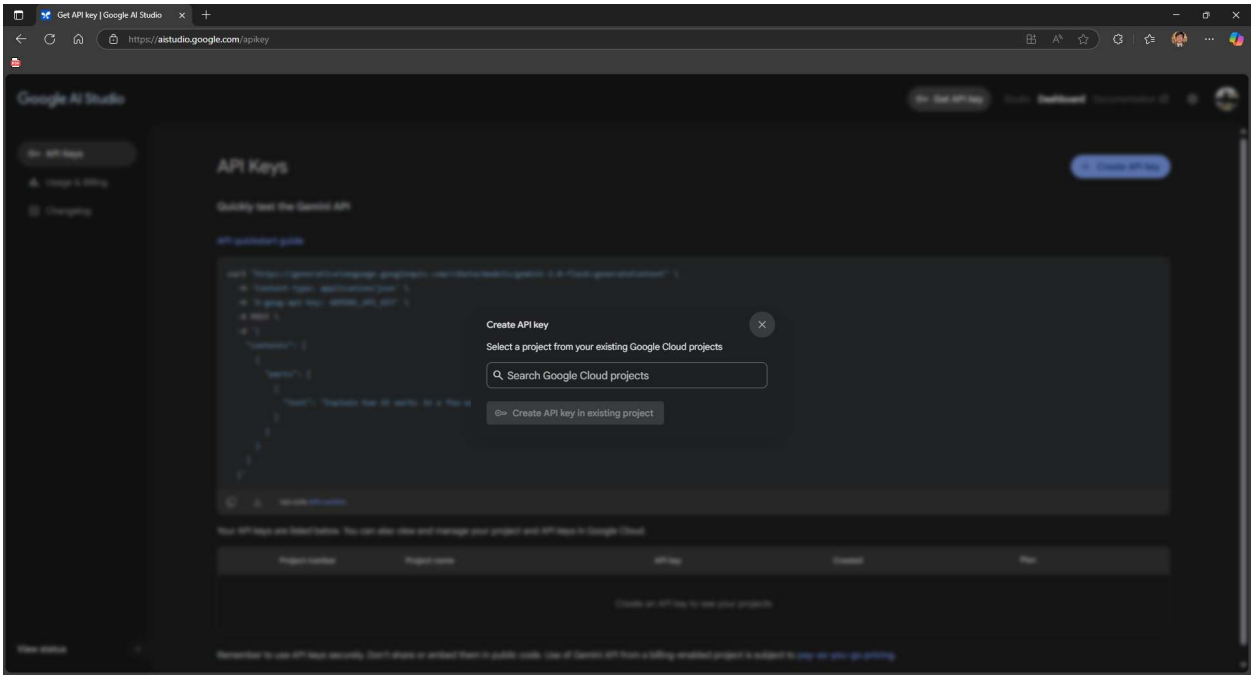

6) Let's create an API key for the newly created project. Select the "Get API key" button.

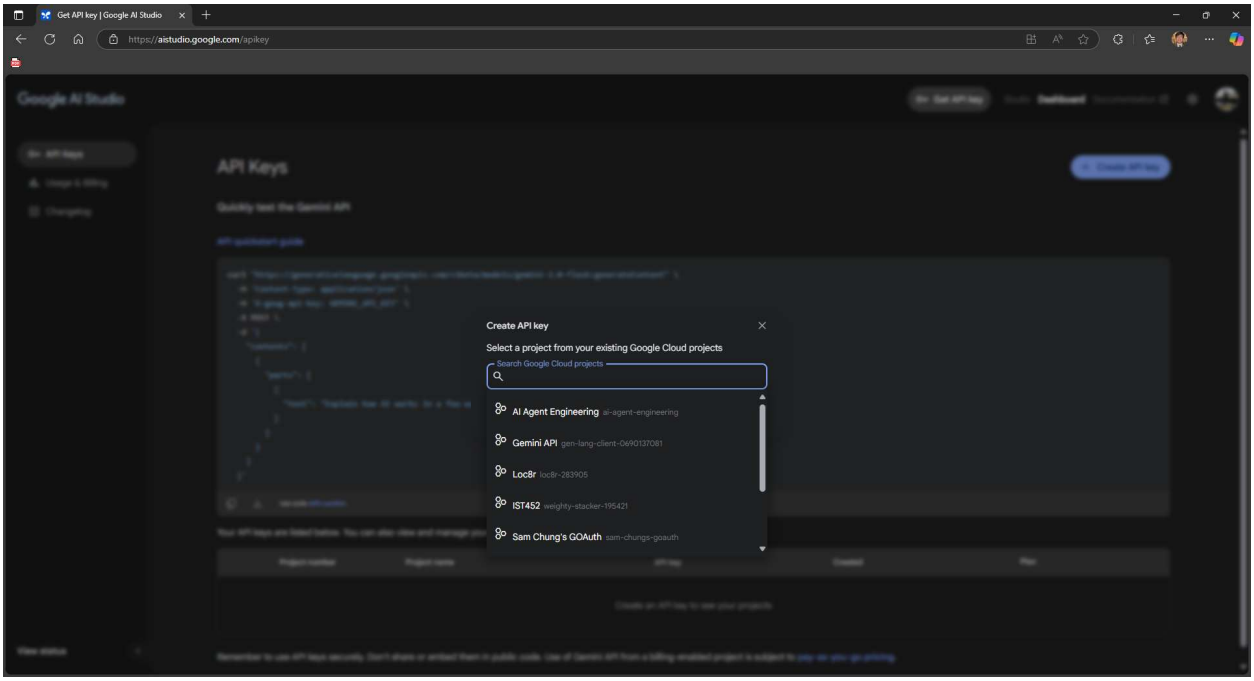


7) Pres the “Create API key” button.

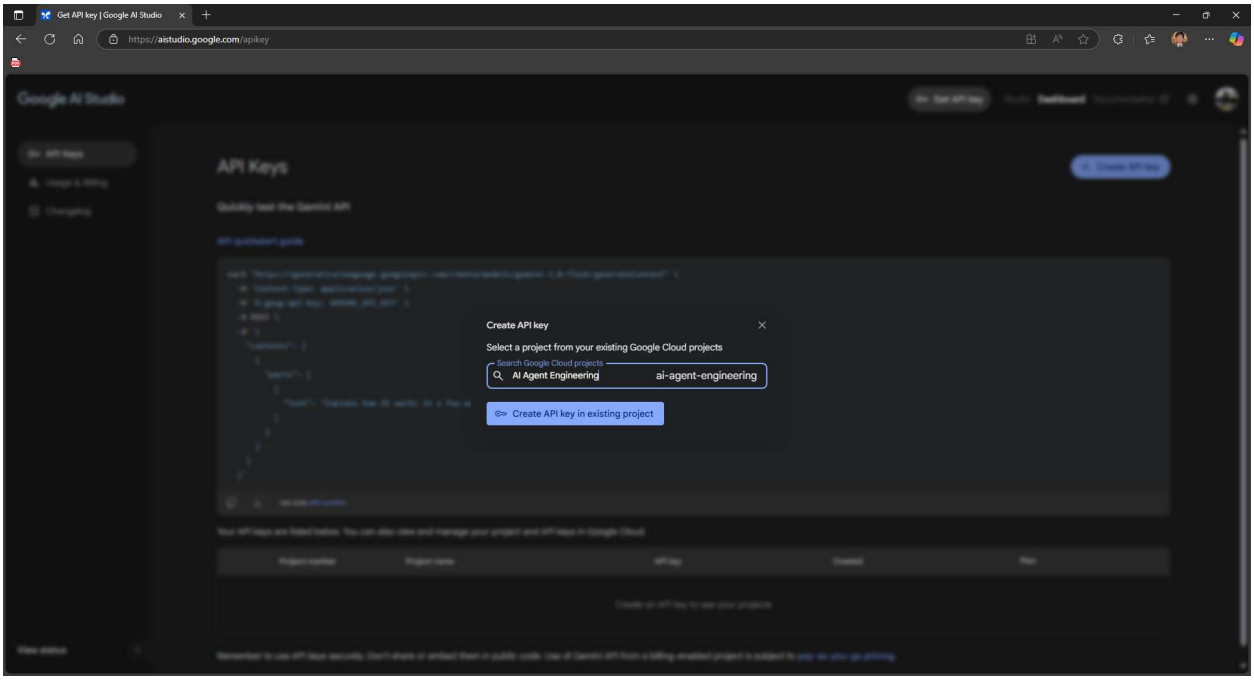



a) Search Google cloud projects.

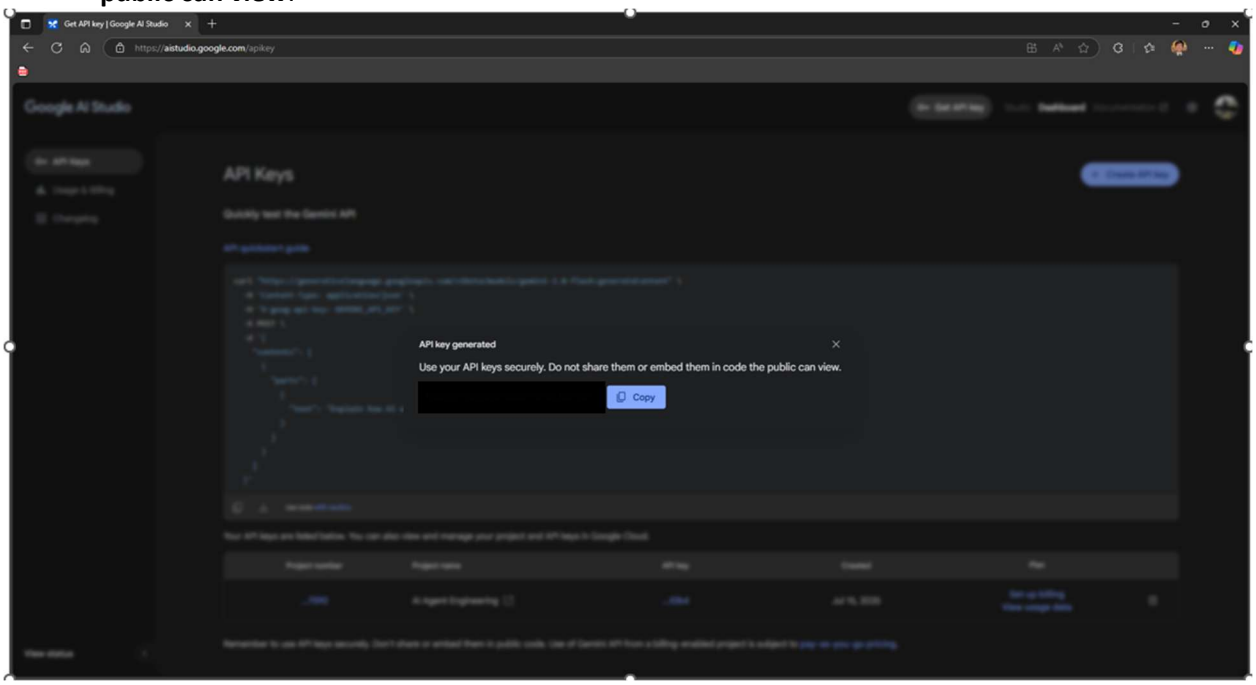




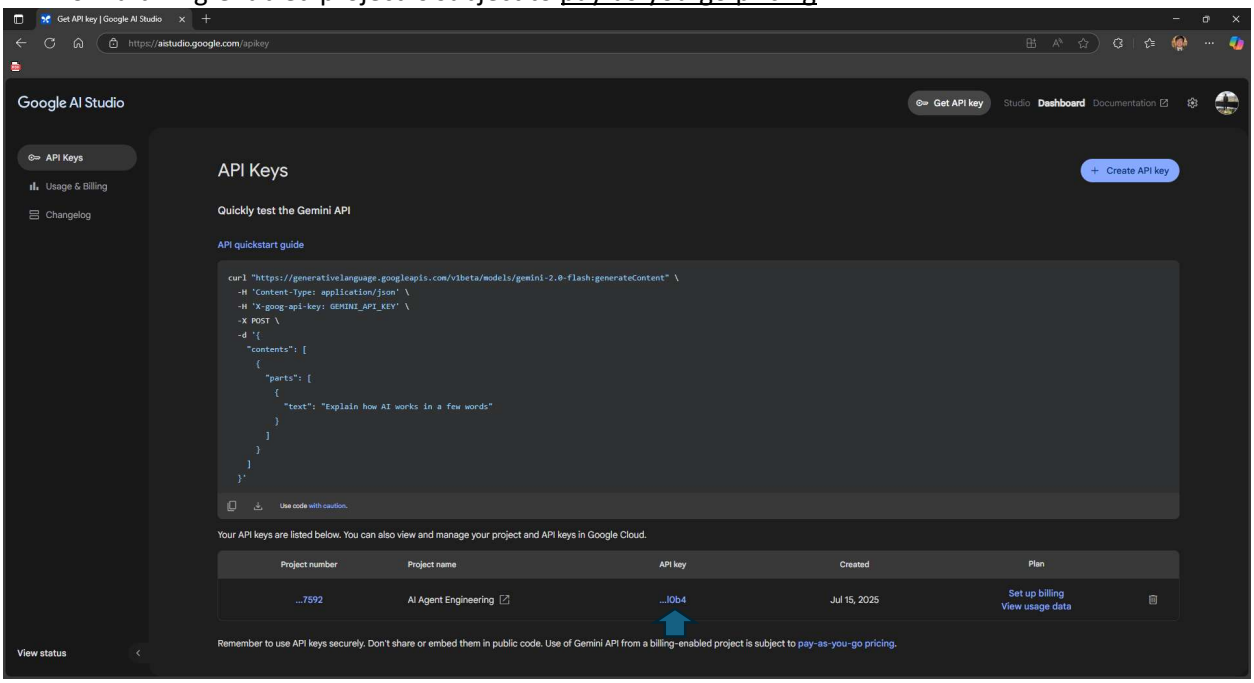

b) Create API key in the existing project.



c) API key generated. Use your API keys securely. **Do not share them or embed them in code the public can view**.




8) You can check the API key from Get API key | Google AI Studio.
Remember to use API keys securely. Don't share or embed them in public code. Use of Gemini API from a billing-enabled project is subject to pay-as-you-go pricing.




7) Take a screenshot 'Section05_root_agent_firstname_lastname.png' by using your first and last name.

**Creating and Updating Your .env file.**

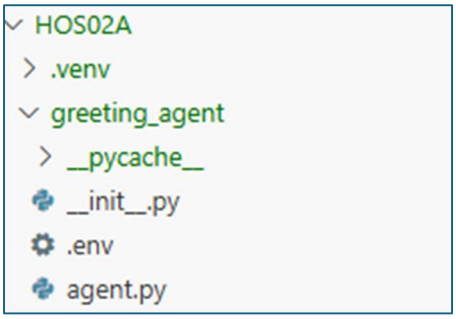
**9)** Open your .env file and type the following:

```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY= PASTE_YOUR_ACTUAL_API_KEY_HERE
```

10) Replace PASTE_YOUR_ACTUAL_API_KEY_HERE with your actual API KEY in the ".env" file.

**Section 6: Running Your Agent using adk web**

1) Using the terminal, navigate to the parent directory of your agent project.

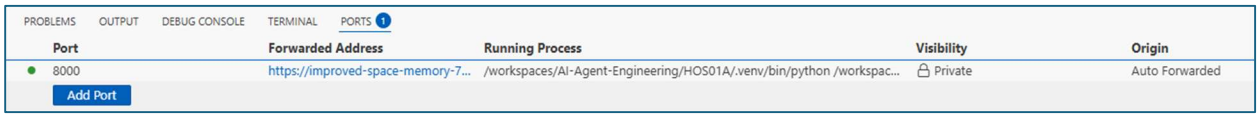



2) Launch the dev UI. In Google ADK (Agent Development Kit), the "dev UI" refers to the built-in, web-based developer user interface that comes with the ADK. It's designed to make the process of developing, testing, and debugging your AI agents much easier and more visual.
**>>> adk web**

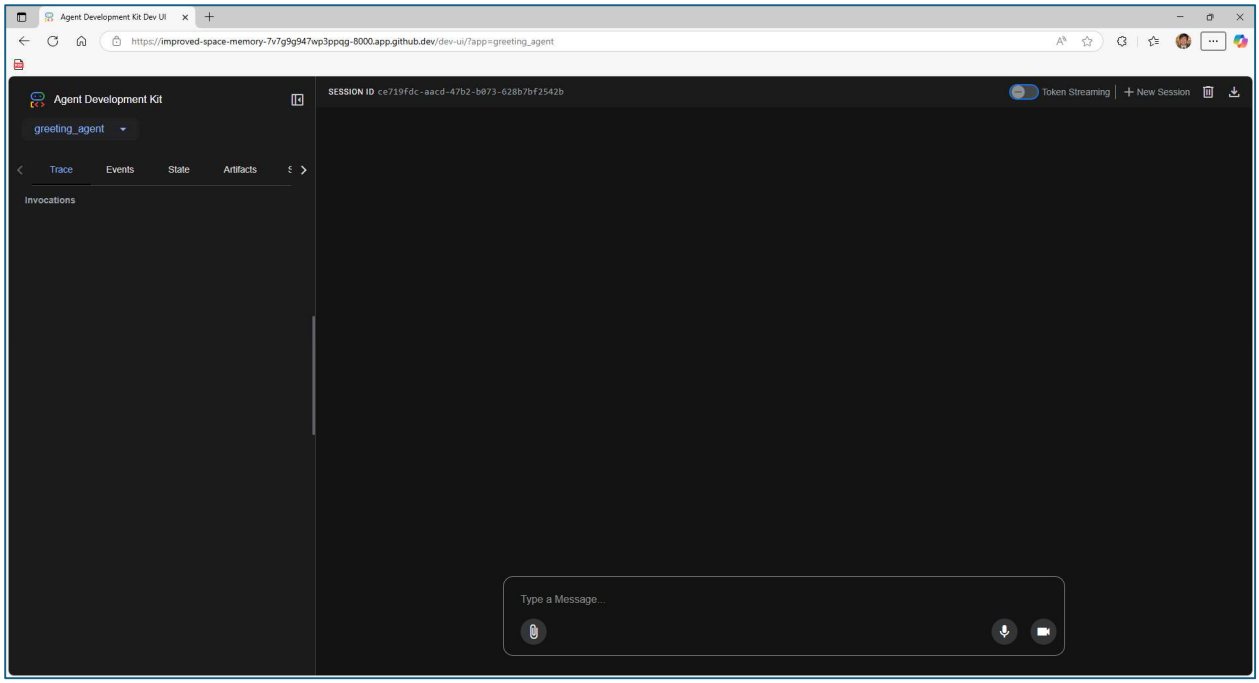
```
(.venv) @samchung0117 ➜/workspaces/AI-Agent-Engineering/HOS02A (main) $ adk web
/workspaces/AI-Agent-Engineering/HOS02A/.venv/lib/python3.12/site-packages/google/adk/cli/fa
d in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/workspaces/AI-Agent-Engineering/HOS02A/.venv/lib/python3.12/site-packages/google/adk/auth/c
imental and may change or be removed in future versions without notice. It may introduce bre
  super().__init__()
INFO:     Started server process [26025]
INFO:     Waiting for application startup.

+-----------------------------------------------------------------------------+
| ADK Web Server started                                                      |
|                                                                             |
| For local testing, access at http://localhost:8000.                         |
+-----------------------------------------------------------------------------+
```

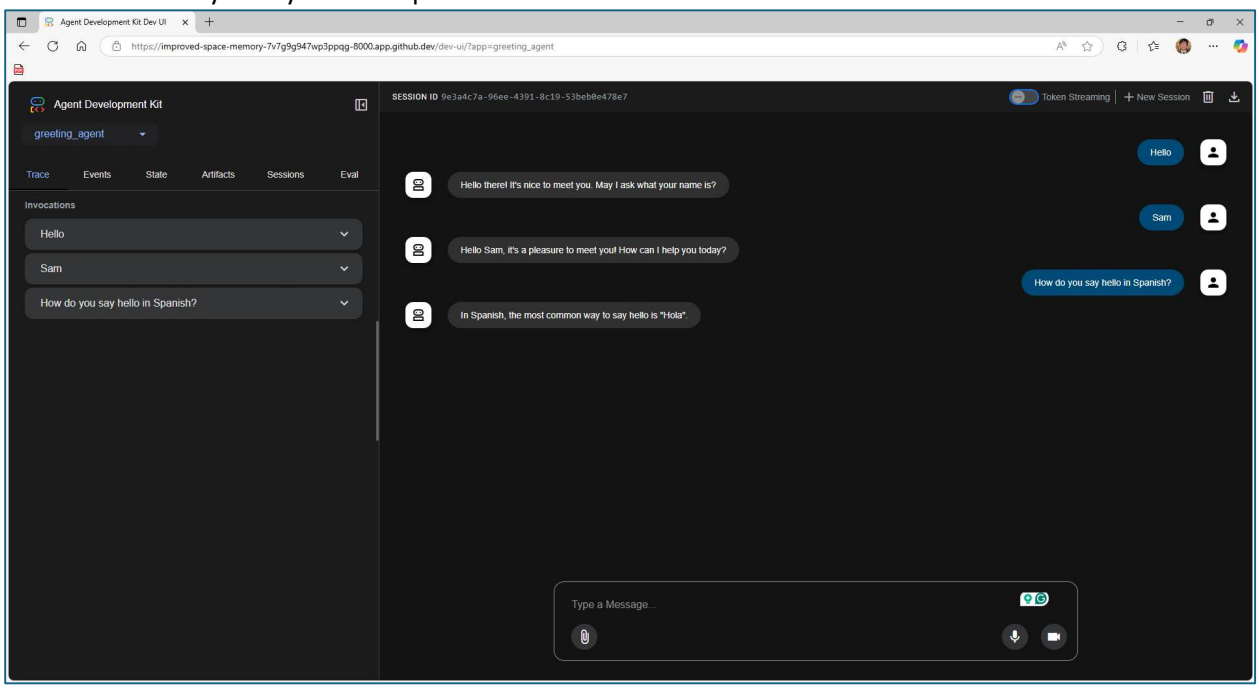
3) Open the URL provided (usually http://localhost:8000 or http://127.0.0.1:8000) directly in your browser.



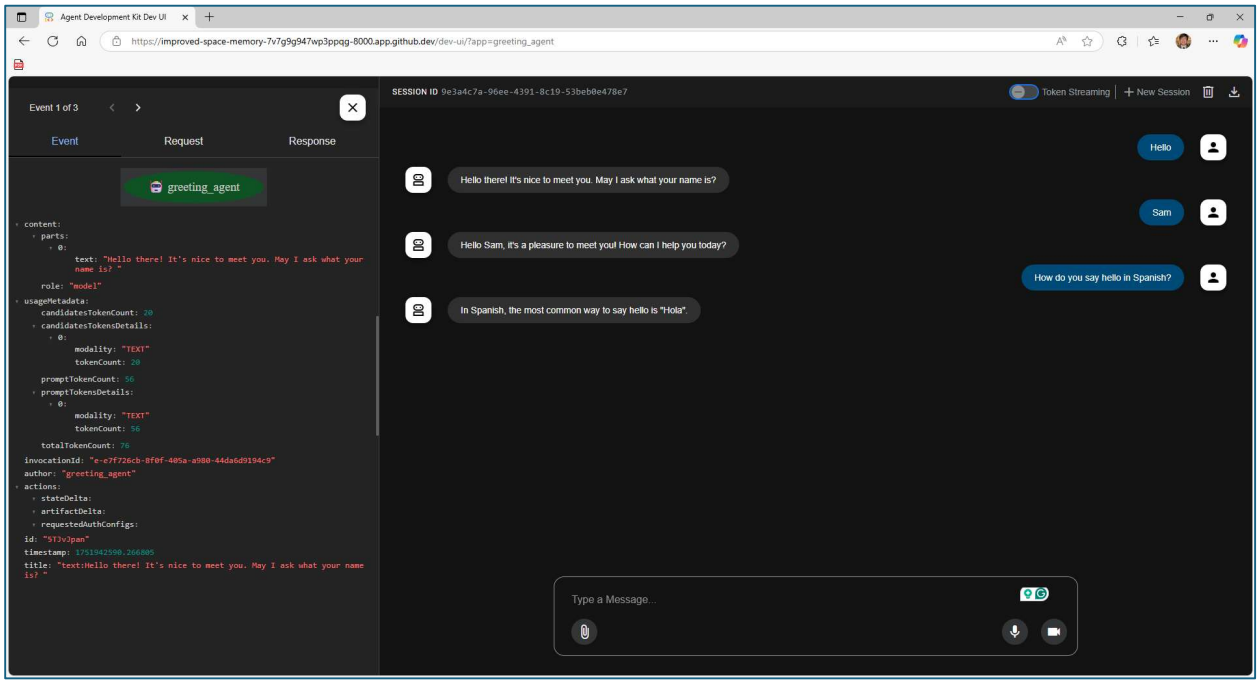

4) In the top-left corner of the UI, you can select your agent in the dropdown. Select "greeting_agent".



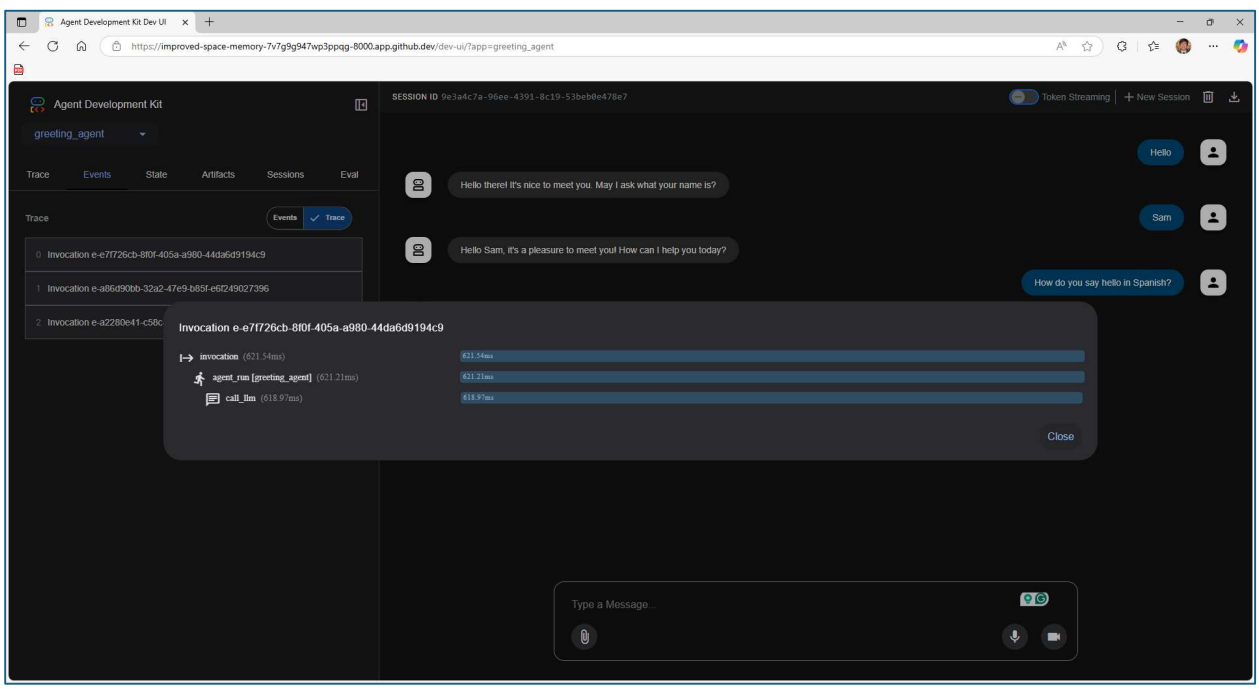

5) Now you can chat with your agent using the following prompts:
   - "How do you say hello in Spanish?"



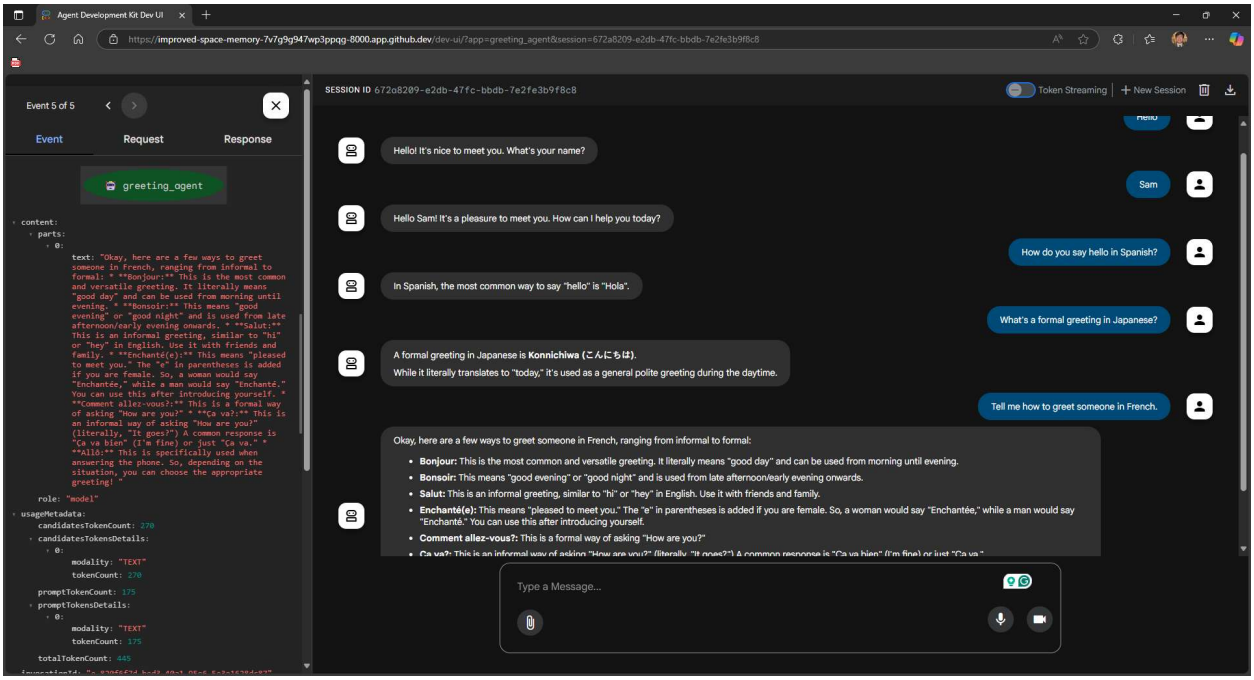
6) By using the Events tab on the left, you can inspect individual function calls, responses and model responses by clicking on the actions. Check all events.




7) On the Events tab, you can also click the Trace button to see the trace logs for each event that shows the latency of each function calls. Check all the invocations.



8) Play with the following prompts in text.
   a) "What's a formal greeting in Japanese?"
   b) "Tell me how to greet someone in French."




9) Take a screenshot 'Section06_Trace_firstname_lastname.png' by using your first and last name.

10) By using MS Copilot, conduct the STRIDE approach.
Take a screenshot 'Section06_STRIDE_firstname_lastname.png' by using your first and last name.

Copy and paste this into your LLM. It is specifically designed to map **STRIDE** to the **2026 OWASP Top 10 for Agentic Applications**.

==========

**Role:** You are a Senior DevSecOps Engineer specializing in Agentic AI architecture.

**Context:** I am developing an AI Agent in **Python** within **GitHub Codespaces**.

- **Agent Goal:** "You are a helpful assistant that greets the user. Ask for the user's name and greet them by name.
- **Tools/Capabilities:** "No tools are used in this project."
- **Environment:** GitHub Codespaces (Linux-based container).

**Task:** Perform a **STRIDE Threat Model** analysis specifically for this agent.

1. **Decompose the System:** Identify the Trust Boundaries (e.g., User -> Agent, Agent -> Tools, Agent -> Environment).
2. **STRIDE Analysis:** For each category (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), identify at least one **Agentic-specific** threat.

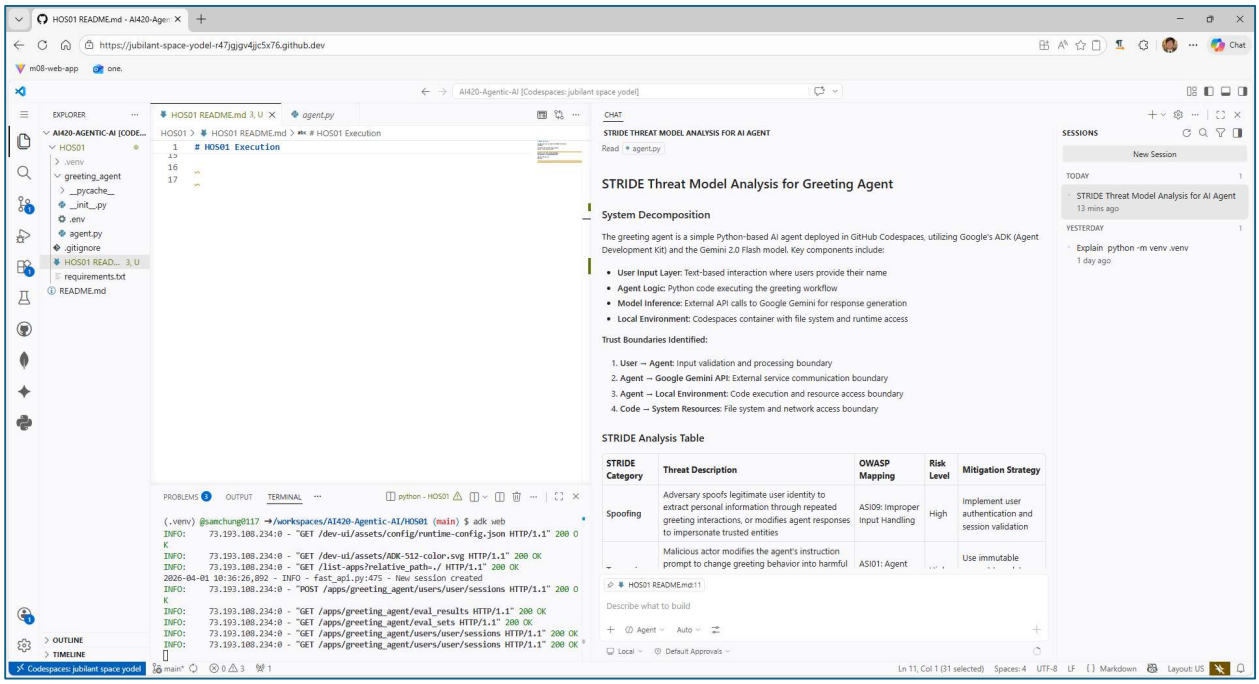
3. **OWASP Alignment:** Map these threats to the [OWASP Top 10 for Agentic AI (2026)](#) (e.g., ASI01: Agent Goal Hijacking, ASI02: Tool Misuse).
4. **Mitigation:** Provide Python-specific code snippets or configuration changes to mitigate the highest risk threats within a Codespaces environment.

**Output Format:** Present the findings on a Markdown table followed by a "Remediation Guide" section.

==========

For example,

Role: You are a Senior DevSecOps Engineer specializing in Agentic AI architecture.
Context: I am developing an AI Agent in Python within GitHub Codespaces.
• Agent Goal: "You are a helpful assistant that greets the user. Ask for the user's name and greet them by name.
• Tools/Capabilities: "No tools are used in this project."
• Environment: GitHub Codespaces (Linux-based container).
Task: Perform a STRIDE Threat Model analysis specifically for this agent.
1. Decompose the System: Identify the Trust Boundaries (e.g., User -> Agent, Agent -> Tools, Agent -> Environment).
2. STRIDE Analysis: For each category (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), identify at least one Agentic-specific threat.
3. OWASP Alignment: Map these threats to the OWASP Top 10 for Agentic AI (2026) (e.g., ASI01: Agent Goal Hijacking, ASI02: Tool Misuse).
4. Mitigation: Provide Python-specific code snippets or configuration changes to mitigate the highest risk threats within a Codespaces environment.
Output Format: Present the findings on a Markdown table followed by a "Remediation Guide" section.



STRIDE Analysis Table

| STRIDE Category | Threat Description | OWASP Mapping | Risk Level | Mitigation Strategy |
|---|---|---|---|---|
| Spoofing | Adversary spoofs legitimate user identity to extract personal information through repeated greeting interactions, or modifies agent responses to impersonate trusted entities | ASI09: Improper Input Handling | High | Implement user authentication and session validation |
| Tampering | Malicious actor modifies the agent's instruction prompt to change greeting behavior into harmful actions (e.g., collecting sensitive data instead of greeting) | ASI01: Agent Goal Hijacking | High | Use immutable prompt templates and code signing |
| Repudiation | Agent lacks comprehensive logging, allowing users or attackers to deny interactions or claim unauthorized access to conversation history | ASI10: Insufficient Monitoring | Medium | Add audit logging for all interactions |
| Information Disclosure | Agent inadvertently discloses user names or conversation context through model hallucinations or insecure storage of session data | ASI07: Sensitive Information Disclosure | High | Implement data minimization and secure storage |
| Denial of Service | Flooding the agent with excessive inputs exhausts API quotas or computational resources, making the greeting service unavailable | ASI08: Unbounded Consumption | Medium | Add rate limiting and resource monitoring |
| Elevation of Privilege | Exploiting agent code vulnerabilities to gain unauthorized access to Codespaces environment or execute arbitrary commands | ASI06: Excessive Agency | High | Restrict agent permissions and sandbox execution |

11) Shut down the server by typing “Control+C”

**Section 8: Pushing Your Work to GitHub**

1) How to submit your work in GitHub

2) How to submit your screenshots in GitHub